

Accès distant en HTTP-IPv4 avec NGINX

Jacques Lemaire & Pierre Nunez

Pierrefeu, avril 2025

Résumé

On se propose ici de montrer comment on peut résoudre le problème d'accès distant à un serveur HTTP-IPv4 situé sur un réseau local, en activant la fonctionnalité « Proxy inverse » du serveur NGINX, lancé comme service OpenWrt sur un routeur IPv4-IPv6, qui définit ce réseau local. L'accès se fera en IPv6, en utilisant l'adresse fixe publique IPv6 attribuée à ce routeur par une box fibre Internet, IPv4-IPv6, fonctionnant en IPv4 CGNAT, donc sans adresse publique IPv4.

Cette étude a été motivée par le pilotage à distance d'une installation photovoltaïque.

Mots clés : HTTP, IPv4, IPv6, CGNAT, Proxy inverse, Proxy direct, NGINX, OpenWrt, Routeur, Réseau local, Accès distant.

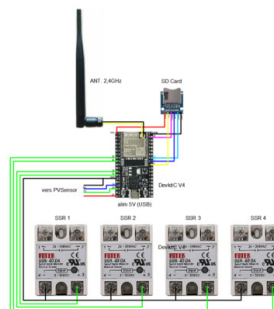
1 Motivation

Le problème à résoudre a été motivé par l'accès à distance aux serveurs HTTP-IPv4 d'un système photovoltaïque domestique, comprenant entre autre :

- Un onduleur hybride photovoltaïque **Fronius GEN24 Primo**¹ :



- Un module DIY² de supervision-commande du système et de pilotage de charges (routeur statique) appelé ici **PVRouter**, basé sur un microcontrôleur ESP32 et des relais solides :



¹ <https://www.fronius.com/fr-fr/france/energie-solaire/installateurs-et-partenaires/produits-et-solutions/solutions-energetiques-residentielles/gen24plus-onduleur-hybride-avec-mode-secours-integre> .

² Do It Yourself.

- Pour des raisons ergonomiques évidentes, la supervision et la commande de tout le système photovoltaïque se fait à partir d'un module également DIY, **PVMonitor** :



Lui aussi basé sur un microcontrôleur ESP32, ce module communique régulièrement avec PVRouter et affiche les flux de puissance électrique qui circulent entre l'onduleur, les panneaux solaires, les consommateurs, le réseau de distribution et la batterie, ainsi que des informations importantes :

- température interne³ et externe ;
- irradiance des panneaux et production PV potentielle ;
- couleur TEMPO EDF du jour et du lendemain ;
- prévisions météo ;
- statistiques de production photovoltaïque ;
- etc.

A l'aide de 4 boutons poussoirs, il permet de commander et de programmer dans le temps l'alimentation des 4 charges importantes du réseau domestique :

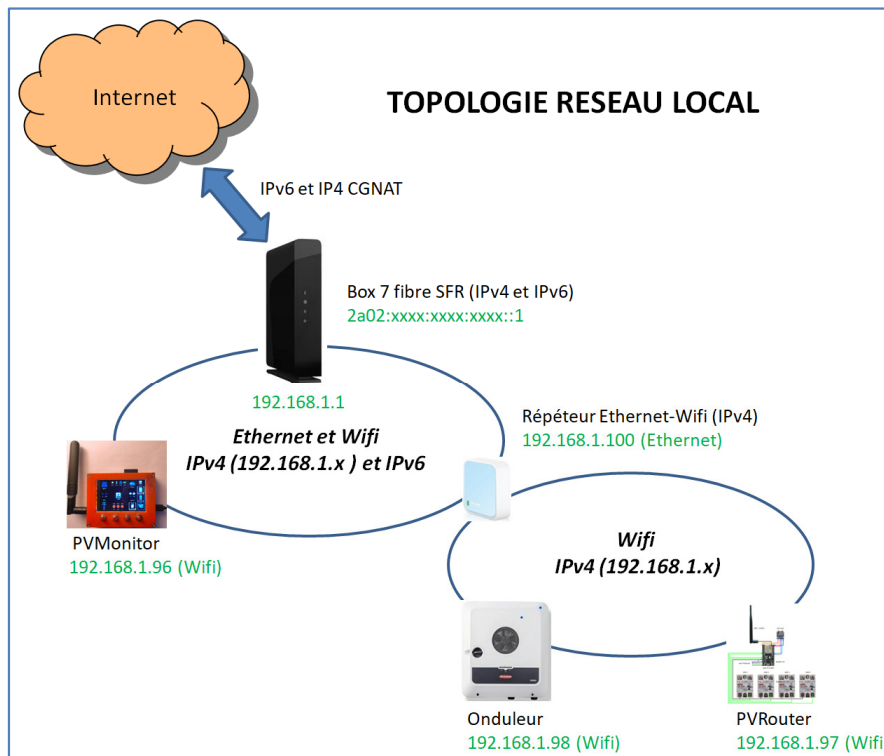
- batterie⁴ ;
- cumulus ;
- PAC ;
- pompe de piscine.

Ces 3 éléments ont été intégrés en Wifi dans le réseau local créé par une **Box 7 SFR**. Pour améliorer les communications, un routeur répéteur **TP-Link TL802N** connecté en Ethernet à la box a été ajouté dans le local où les 2 premiers éléments sont installés. La box fonctionne en IPv4 et IPv6 (cf. § 2.1) ; elle est connectée à Internet en IPv6 et IPv4 CGNAT (cf. § 2.2) et elle attribue des adresses IPv4 à tous les éléments précédents⁵ :

³ Qu'il mesure lui-même.

⁴ Charge forcée ou décharge bloquée.

⁵ Ils ne fonctionnent qu'en IPv4.



Les 3 éléments du système photovoltaïque communiquent entre eux en Wifi, à l'aide de requêtes HTTP-IPv4 ou Modbus TCP⁶ de la manière suivante :

PVMonitor ⇔ PVRouter ⇔ Onduleur

Sans trop rentrer dans les détails, PVMonitor formule des requêtes **HTTP GET** au serveur de PVRouter sur le port 8080, avec des URL de la forme⁷ :

<http://192.168.1.97:8080/data?Temperature=25&Battery=2&Boiler=2&HPump=2&SPPump=2>

Quand il les reçoit, PVRouter exécute les commandes d'alimentation des charges précitées⁸ puis retourne à PVMonitor les informations nécessaires pour rafraîchir son affichage, en format **JSON**⁹. On peut le constater sur un PC intégré dans ce réseau local, en indiquant cette URL dans un navigateur Internet :

```
{
  "pA": 433.43,
  "pB": 432.54,
  "pC": -0.89,
  "pD": 0,
  "pOP": 0,
  "SOC": 26.2,
  "tWat": 29.8,
  "tOut": 0,
  "lSun": 0,
  "pPot": 0,
  "vBat": 0,
  "dJC": 3,
  "dJ1C": 3,
  "dJ1I": 19,
  "dJ1I": 4,
  "mR": 0,
  "dJR": 0,
```

⁶ <https://modbus.org/specs.php> par exemple.

⁷ 0=>OFF, 1=>ON, 2=>AUTO.

⁸ 0=>OFF, 1=>ON, 2=>AUTO.

⁹ Java Script Object Notation

```

    "dJ1R": 0,
    "dSPP": 0
}

```

De la même manière, PVRouter questionne l'onduleur, avec l'URL :

http://192.168.1.98/solar_api/v1/GetPowerFlowRealtimeData.fcgi

Ce dernier retourne des informations sur son état¹⁰, là encore en format JSON :

```

{
  "Body" : {
    "Data" : {
      "Inverters" : {
        "1" : {
          "Battery_Mode" : "normal",
          "DT" : 1,
          "E_Day" : null,
          "E_Total" : 1757314.723888889,
          "E_Year" : null,
          "P" : 2182.44970703125,
          "SOC" : 97.599999999999994
        }
      },
      "SecondaryMeters" : {},
      "Site" : {
        "BackupMode" : false,
        "BatteryStandby" : true,
        "E_Day" : null,
        "E_Total" : 1757314.723888889,
        "E_Year" : null,
        "Meter_Location" : "grid",
        "Mode" : "bidirectional",
        "P_Akku" : 2045.85791015625,
        "P_Grid" : 42.200000000000003,
        "P_Load" : -2224.6497070312498,
        "P_PV" : 238.76676940917969,
        "rel_Autonomy" : 98.103072143599874,
        "rel_SelfConsumption" : 100.0
      },
      "Smartloads" : {
        "Ohmpilots" : {}
      },
      "Version" : "12"
    }
  },
  "Head" : {
    "RequestArguments" : {},
    "Status" : {
      "Code" : 0,
      "Reason" : "",
      "UserMessage" : ""
    },
    "Timestamp" : "2023-02-22T15:36:16+00:00"
  }
}

```

PVRouter envoie des requêtes Modbus TCP en IPv4¹¹ à l'onduleur pour activer la charge forcée de la batterie ou bloquer sa décharge; il active ou désactive ses relais solides pour commander l'alimentation ou la coupure d'alimentation des autres charges importantes du réseau domestique.

Il est important de bien comprendre que tout ceci ne met en œuvre que des communications entre postes situés sur un même réseau local. Mais comment procéder si on veut maintenant le faire à distance ?

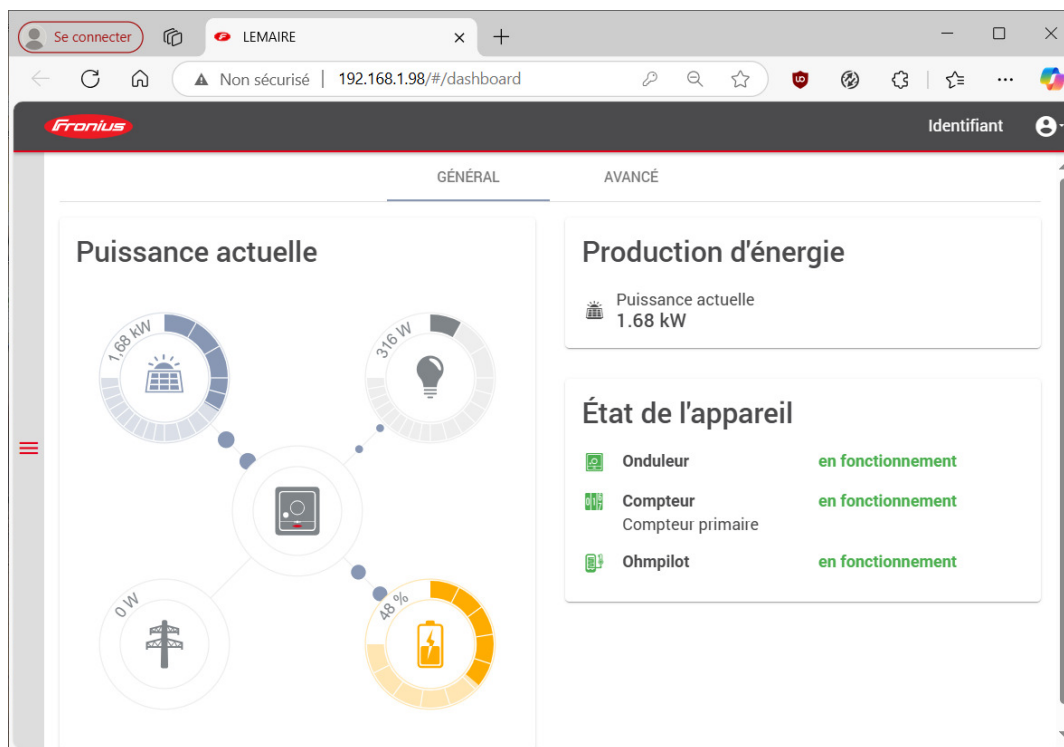
¹⁰ <https://www.fronius.com/fr-fr/france/energie-solaire/installateurs-et-partenaires/caracteristiques-techniques/tous-les-produits/surveillance-des-installations/interfaces-ouvertes/fronius-solar-api-json->

¹¹ <https://www.fronius.com/fr-fr/france/energie-solaire/installateurs-et-partenaires/caracteristiques-techniques/tous-les-produits/surveillance-des-installations/interfaces-ouvertes/fronius-solar-api-json->

Pour illustrer la chose, considérons par exemple l'accès direct à l'interface Web de l'onduleur. On peut le faire localement à partir d'un PC intégré dans le réseau local précédent avec l'URL :

<http://192.168.1.98>

Elle conduit à la page Web suivante :



Cette page affiche les flux de puissance électrique circulant entre l'onduleur, les panneaux solaires, les consommateurs, le réseau de distribution et la batterie, ainsi que la charge de cette dernière.

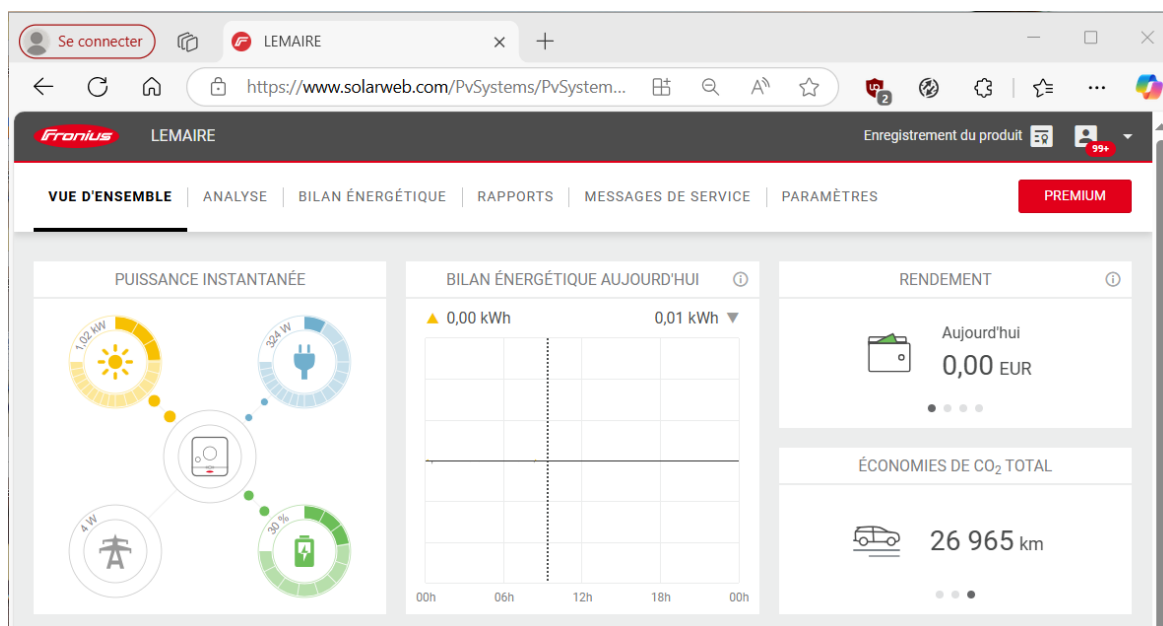
Mais, en cliquant sur , on peut également accéder à une arborescence de menus permettant de paramétrer et donc de piloter l'onduleur ; par exemple la charge-décharge de la batterie :

The screenshot shows the 'Gestion de la batterie' (Battery Management) settings page. It includes a 'Battery SoC Settings' section with a dropdown for 'Mode de limitation de l'état de charge' set to 'Auto'. Below this are input fields for 'Etat de charge minimal' (5%), 'Etat de charge maximal' (100%), 'Capacité de réserve' (5%), and 'Limite d'avertissement de l'état de charge' (5%). There is a toggle for 'Battery charging from other sources' which is turned on, and two radio buttons for 'Autoriser la charge de batterie à partir du réseau public' (selected) and 'Autoriser la charge de batterie à partir d'autres générateurs sur le réseau domestique'. The bottom section, 'Contrôle programmé de la batterie', contains a table with two rows of scheduled charging and discharging settings.

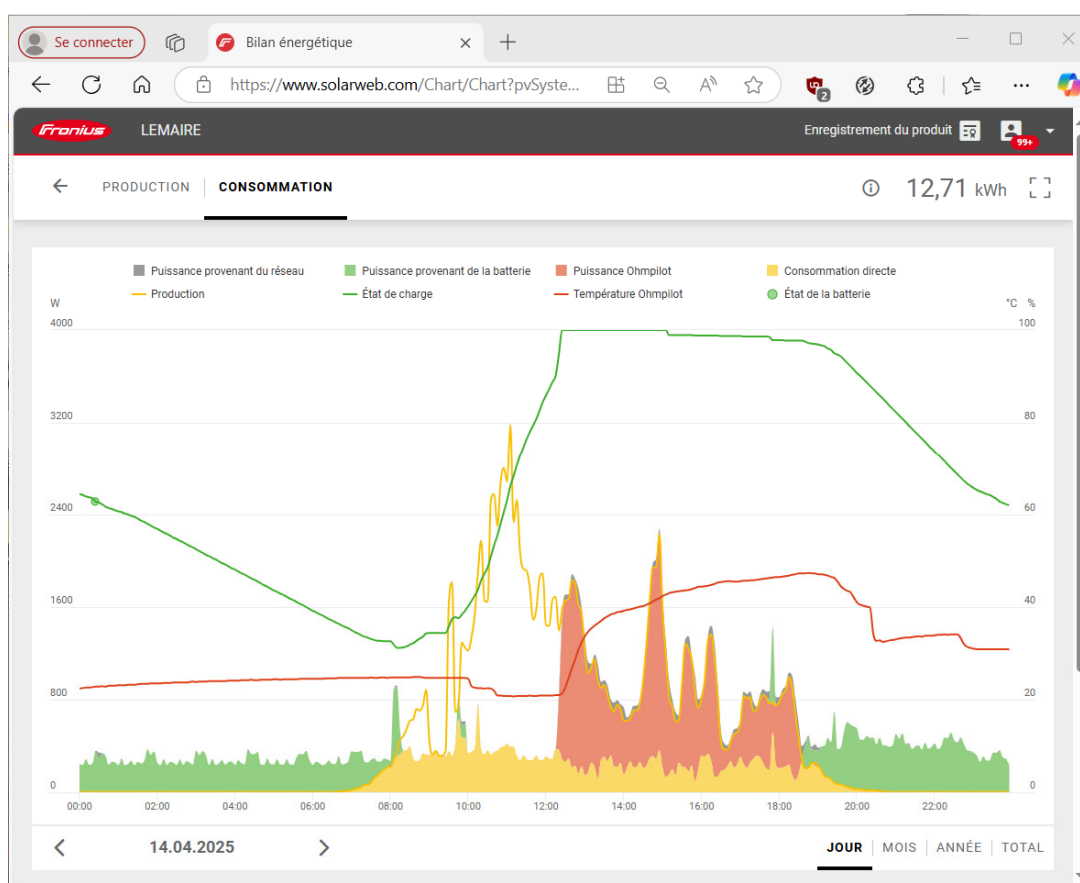
Contrôle programmé de la batterie			
1. Puissance de charge min.	1000 W	00:00 – 06:00	Lu Ma Me Je Ve Sa Di
2. Puissance de décharge max.	0 W	00:00 – 06:00	Lu Ma Me Je Ve Sa Di

Pour y accéder à distance, une solution classique existe : elle consiste à passer par un serveur public externe (**proxy inverse**), qui communique avec le serveur privé (l'onduleur ici) via un tunnel Internet.

Dans notre cas, on peut utiliser le serveur SOLAR WEB de Fronius¹² qui retourne la page suivante, après identification :

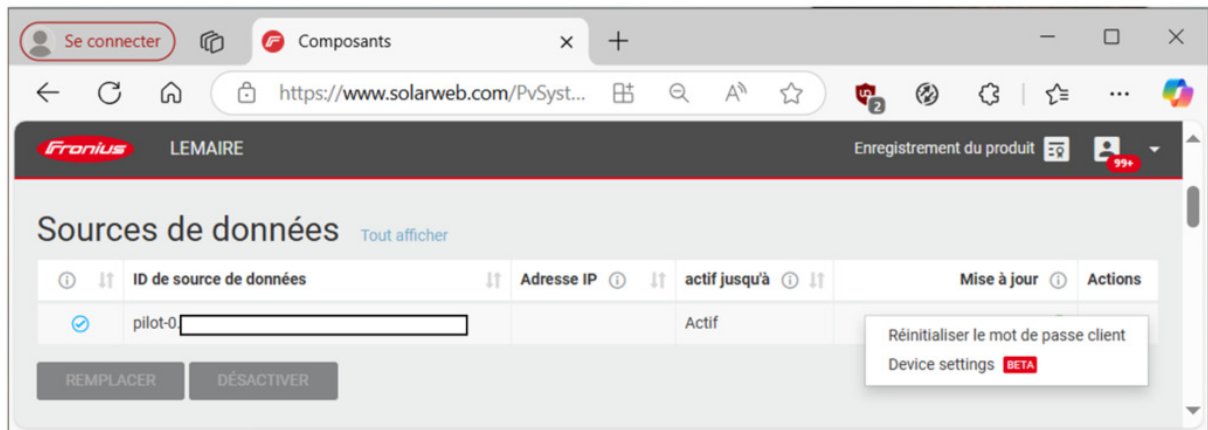


Là encore les flux de puissance électrique sont affichés, ainsi que bilans de production ou de consommation ; par exemple, l'item de menu **BILAN ENERGETIQUE > CONSOMMATION** affiche :



Mais surtout, en utilisant l'item de menu **PARAMETRES > COMPOSANTS > Actions > Device Settings** récemment ajouté :

¹² www.solarweb.com.

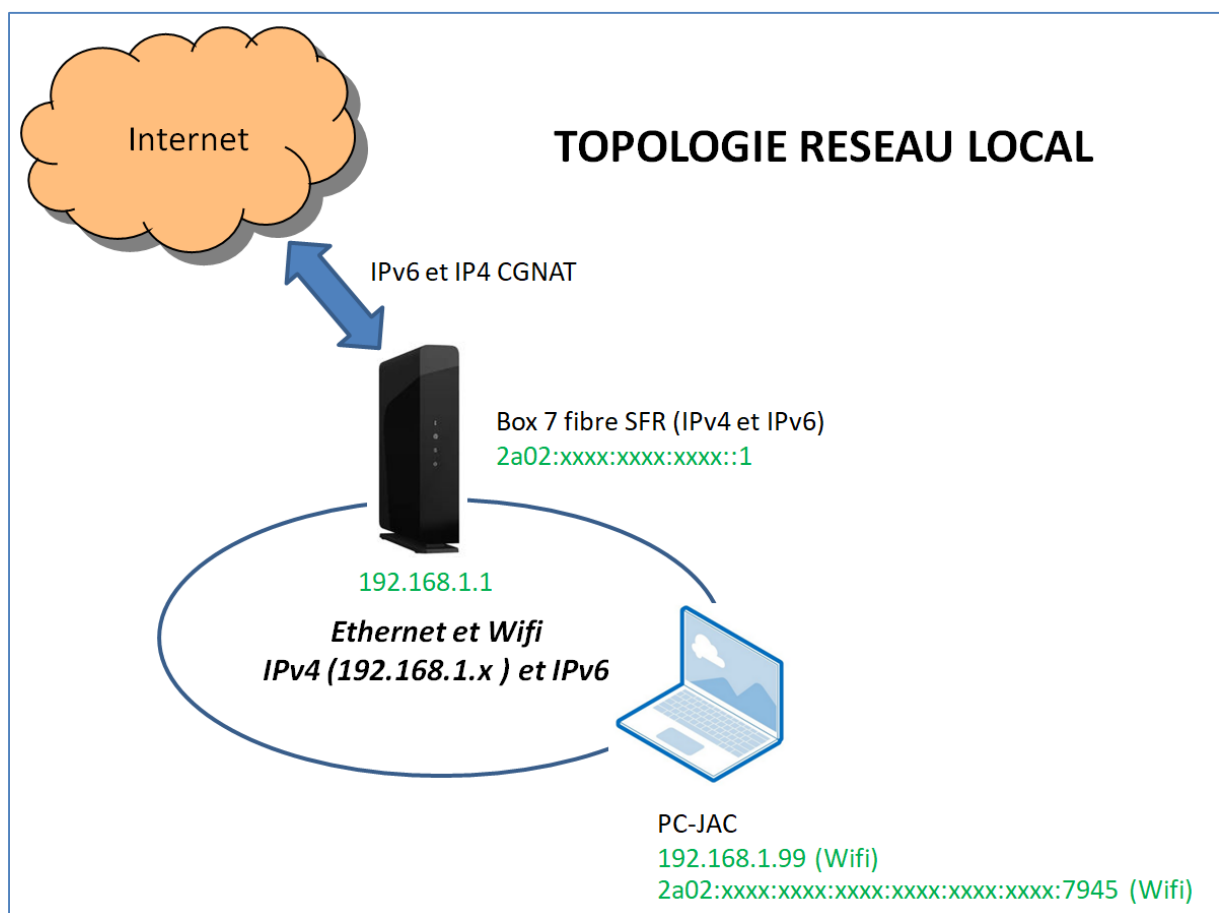


on est redirigé vers le site Web de l'onduleur. Grâce à cette technique on peut donc maintenant y accéder à distance.

Mais que faire maintenant quand un tel proxy inverse n'existe pas, comme c'est le cas par exemple pour notre module PVRouter qui, rappelons-le, expose un serveur HTTP IPv4 sur le port 8080 ?

Nous allons là encore construire un proxy inverse, avec une adresse publique, mais cette fois situé dans le réseau local défini par la box.

Pour simplifier la présentation de cette solution, on utilisera un PC intégré dans ce réseau local, fonctionnant en IPv4 et IPv6 (au départ), et sur lequel on fera fonctionner un serveur Web beaucoup moins complexe que celui de PVRouter :



2 Concepts et outils de base

2.1 Adressage IP

Pour pouvoir communiquer entre eux, les postes du réseau Internet utilisent des adresses IP¹³ classiquement attribuées par les FAI¹⁴ ou les box.

Actuellement 2 normes coexistent :

- **IPv4** où les adresses sont définies par une suite de 4 nombres entiers compris entre 0 et 255 ; par exemple :

192.164.8.10

On peut les coder sur 32 bits et il y a donc $2^{32} \approx 4,3$ milliards d'adresses IPv4 possibles au maximum.

- **IPv6** où les adresses sont définies par une suite de 8 nombres entiers compris entre 0 et 65535 ; par exemple :

10821.33096.36096.19474.0.0.50407.45063

Ici on préfère souvent utiliser leurs écritures hexadécimales, séparées par ':' :

2a45:8148:8d00:4c12:0000:0000:c4e7:b007

en omettant les 0 pour simplifier :

2a45:8148:8d00:4c12::c4e7:b007

On peut les coder sur 128 bits et il y a donc cette fois $2^{128} \approx 340$ milliards de milliards de milliards d'adresses IPv6 possibles au maximum.

Pour pallier le nombre insuffisant d'adresses IPv4 et sécuriser leur accès, les postes d'Internet sont généralement regroupés en réseaux locaux où chaque poste reçoit une adresse **privée** IPv4, très souvent de la forme :

192.168.x.y

On peut donc y accéder facilement en IPv4 à partir de n'importe quel autre poste situé sur le même réseau local¹⁵ mais, dans le cas contraire¹⁶, ce n'est pas possible, sauf si on arrive à passer par un proxy-inverse, disposant d'une adresse **publique** et sachant communiquer avec le poste précédent. C'est par exemple ce qu'on fait pour accéder au site Web de l'onduleur Fronius GEN24, en passant par **www.solarweb.com**.

Par contre, le problème ne se posera plus en IPv6 car tous les postes disposent, pour l'instant du moins, d'une adresse **publique** et on peut donc y accéder directement : c'est ce qu'on va mettre en application ici.

¹³ Internet Protocol.

¹⁴ Fournisseurs d'Accès à Internet.

¹⁵ Accès **local**.

¹⁶ Accès **distant**.

2.2 Box Internet

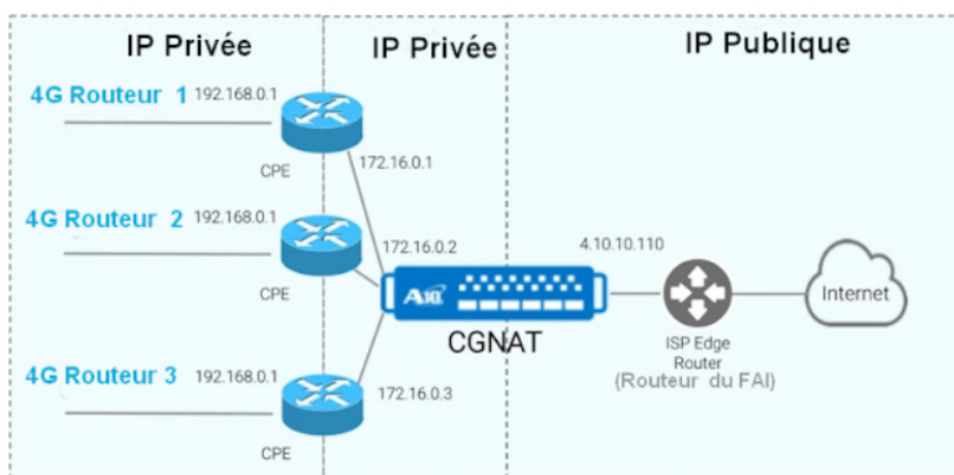
Dans une installation domestique, c'est elle qui va souvent permettre de construire le réseau local, en exposant des services DHCP¹⁷ qui attribueront des adresses IPv4 et IPv6 aux postes connectés, ainsi que les réglages classiques nécessaires pour accéder à Internet :

- Adresse locale IP de la passerelle (souvent la box elle-même).
- Masque de sous-réseau.
- Adresse(s) IP d'un (ou plusieurs) serveur(s) DNS.

Classiquement, en IPv4, la box reçoit l'adresse privée 192.168.1.1 dans le réseau local qu'elle implémente, avec comme masque de sous-réseau 255.255.255.0 : ceci signifie qu'elle attribuera des adresses privées de la forme 192.168.1.x, par exemple 192.168.1.99 pour le PC considéré précédemment. On peut aussi fournir son adresse privée comme DNS car elle redirige les requêtes à de tels serveurs.

Avant les box n'implémentaient que le protocole IPv4 mais recevaient une adresse publique IPv4, fixe ou variable. Quitte à passer par un serveur **DynDNS**¹⁸ et implémenter un client DynDNS sur la box dans le cas variable, puis définir des règles **NAT-PAT**¹⁹ sur la box et son pare-feu, on pouvait faire en IPv4 des accès distants aux serveurs de son réseau local.

Maintenant les box se connectent souvent à Internet en **IPv4 CGNAT**²⁰ :



Dans cette solution temporaire en attendant l'utilisation systématique d'IPv6, le FAI²¹ de la box crée des réseaux privés regroupant un grand nombre de box clients derrière un routeur CGNAT qui seul reçoit une adresse publique IPv4²². La communication se fait là encore via des règles NAT, mais on ne peut plus ajouter de telles règles sur le routeur CGNAT, pour réaliser des accès distants vers un poste du réseau local défini par une de ces box, comme on le faisait avant sur ces dernières.

Par contre les box peuvent maintenant travailler aussi en IPv6 et reçoivent dans ce cas une adresse IPv6 publique.

¹⁷ Dynamic Host Configuration Protocol.

¹⁸ NO-IP (<https://www.noip.com/>) par exemple.

¹⁹ NAT = Network Address Translation, PAT = Port Address Translation.

²⁰ Carrier Grade NAT. Cf. par exemple <https://abcreseau.blogspot.com/2020/08/du-nat-au-cgnat.html> ou <https://www.rapidseedbox.com/fr/blog/cgnat> pour des explications.

²¹ Fournisseur d'Accès Internet.

²² On peut la faire afficher en appelant www.monip.org

Dans notre cas, il s'agit d'une box 7 fibre SFR²³ connectée à Internet via un ONT²⁴ :



A partir d'un poste de son réseau local, on peut accéder à son paramétrage, en utilisant l'URL <http://192.168.1.1> dans un navigateur Internet. Si on le fait en tant qu'administrateur²⁵, on peut constater les choses suivantes :

- Item de menu **Home** :

Adresse IPv6	2a02: ::1/64
--------------	-----------------------------------

Dans la notation IPv6²⁶ ceci correspond à :

2a02:xxxx:xxxx:xxxx:0000:0000:0000:0001

et le /64 précise qu'elle attribuera des adresses publiques IPv6 de la forme :

2a02: xxxx:xxxx:xxxx:yyyy:yyyy:yyyy:yyyy

aux postes du réseau local qu'elle définit.

- Item de menu **Etat > Internet** :

Internet IPv4	
Statut de l'accès	●●● Opérationnel
Temps d'accès	34 j 23 h 14 min. 28 sec.
Protocole	Carrier-grade NAT

Cette box peut donc bien communiquer avec Internet en IPv4 et IPv6 et elle peut attribuer ces 2 types d'adresses aux postes du réseau local qu'elle définit : c'est ce qu'on peut par exemple constater sur un PC sous Windows 10, faisant partie de ce réseau local et connecté en Wifi à la box, en tapant **IPCONFIG** dans l'invite de commandes :

²³ Modèle NB6VAC-FXC-r0 exactement.

²⁴ Optical Network Termination.

²⁵ Appuyer sur le bouton WPS pendant 5s dans notre cas.

²⁶ <https://www.ionos.fr/digitalguide/serveur/know-how/quels-sont-les-avantages-de-ipv6/>

```

Carte réseau sans fil Wi-Fi :
Suffixe DNS propre à la connexion. . . . :
Adresse IPv6. . . . . : 2a02: [redacted] :7945
Adresse IPv6. . . . . : 2a02: [redacted] :2563
Adresse IPv6 temporaire. . . . . : 2a02: [redacted] :e187
Adresse IPv6 de liaison locale. . . . . : fe80::e6f3:b4c0:6af4:a803%8
Adresse IPv4. . . . . : 192.168.1.99
Masque de sous-réseau. . . . . : 255.255.255.0
Passerelle par défaut. . . . . : fe80::e65d:51ff:fe94:6480%8
                               192.168.1.1

```

Ce PC reçoit donc ici l'adresse publique IPv6²⁷ :

2a02:xxxx:xxxx:xxxx:xxxx:xxxx:7945

et l'adresse privée IPv4

192.168.1.99

Remarque

Ces adresses sont attribuées par les services DHCP box à chaque poste du réseau local, en tenant compte de leur adresse MAC²⁸, comme on peut le constater dans le paramétrage de la box, pour le PC précédent :

- Item de menu **Home > Réseau v4 > Général** :

Postes connectés					
...					
8	7c:[redacted]bd	PC-JAC	192.168.1.99	Wifi 5GHz	

- Item de menu **Home > Réseau v6 > General** :

Postes connectés					
...					
3	7c:[redacted]bd	2a02:[redacted]:7945 (PC-JAC)	Wifi 5GHz		

Afin que ces adresses ne varient pas, il suffit de les rendre **statiques** :

- Item de menu **Home > Réseau v4 > DHCP** :

Adresses statiques	
...	
192.168.1.99	7c:[redacted]bd

- Item de menu **Home > Réseau v6 > DHCP** :

Adresses statiques	
...	
2a02:[redacted]:7945	7c:[redacted]bd

²⁷ On pourra se reporter à la référence précédente pour la signification des autres adresses IPv6 ici attribuées.

²⁸ Media Access Control

2.3 Serveur HTTP local

Il simulera une situation analogue à celle décrite dans le §1 pour le serveur HTTP de **PVRouter**, mais beaucoup plus simple, où :

- Entrée : commande **GET HTTP** avec paramètres.
- Sortie : page Web dynamique créée à partir de ces derniers.

En outre, pour bien montrer les problèmes d'accès selon le protocole IP utilisé, il fonctionnera en **IPv4** ou **IPv6**.

On va ici proposer une solution implémentée avec un PC sous Windows 10 ou 11, mais on peut choisir bien d'autres configurations matérielles ou logicielles.

Elle est mise en œuvre très simplement, en utilisant le langage de programmation **Python**²⁹ :

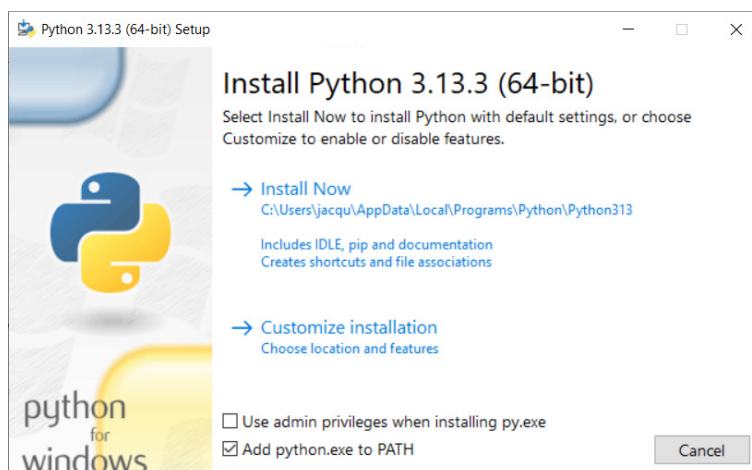


et principalement sa bibliothèque **http.server**.

Pour l'installer sur un PC fonctionnant sous Windows 10 dans notre cas, il suffit de télécharger :

python-3.13.3-amd64.exe³⁰

et de l'exécuter, en cliquant sur Install Now, après avoir coché la case **Add python.exe to PATH** :



2.3.1 Server Web en IPv4

Pour implémenter un serveur Web fonctionnant en IPv4 sur le port 8080 et qui réponde aux objectifs fixés, on va utiliser le programme Python suivant^{31 32} :

²⁹ <https://www.python.org/>

³⁰ Dernière version pour Windows sur <https://www.python.org/downloads/>

³¹ Adapté de <https://pythonbasics.org/webserver/> et <http://stackoverflow.com/questions/24580613/python-start-http-server-in-code-create-py-to-start-http-server>

```

# server4.py
#
# Enable a Web server in IPv4 on the 8080 port which decodes 2 parameters
# in the URL and return their values in a dynamic HTML page.
# Call : http://<IPv4 PC address>:8080/?param1=<value1>&param2=<value2>

from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs

serverPort = 8080

def printHTML(self, s):
    self.wfile.write(bytes(s, "utf-8"))
    self.wfile.write(bytes('\n', "utf-8"))

class MyServer(BaseHTTPRequestHandler):
    def do_GET(self):
        # Decode parameters
        url_parts = urlparse(self.path)
        query_params = parse_qs(url_parts.query)
        val1 = query_params.get('param1', [''])[0]
        val2 = query_params.get('param2', [''])[0]

        # Response with dynamic web page
        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()
        printHTML(self, "<!DOCTYPE html>")
        printHTML(self, "<html>")
        printHTML(self, "    <body>")
        printHTML(self, "        <p>param1 : %s</p>" % val1)
        printHTML(self, "        <p>param2 : %s</p>" % val2)
        printHTML(self, "    </body>")
        printHTML(self, "</html>")

webServer = HTTPServer(("", serverPort), MyServer)
print("Serveur actif sur le port :", serverPort)
webServer.serve_forever()

```

Il définit un script serveur, **do_GET**, qui répond à une requête **HTTP GET** de la forme :

http://<adresse IPv4 du PC>:8080/?param1=<valeur1>¶m2=<valeur2>

Il décode les valeurs des 2 paramètres et retourne une page Web dynamique affichant ces valeurs³³.

Une bonne façon de procéder pour le créer et l'exécuter consiste à utiliser ici l'outil **VS Code**³⁴ :



³² On peut créer très simplement un serveur Web élémentaire en Python avec la commande :

```

C:\> Invite de commandes
Microsoft Windows [version 10.0.19045.5608]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\jacqu>python -m http.server

```

mais sans pouvoir ici effectuer les traitements visés coté serveur. Par contre ce serveur fonctionne en IPv4 et IPv6.

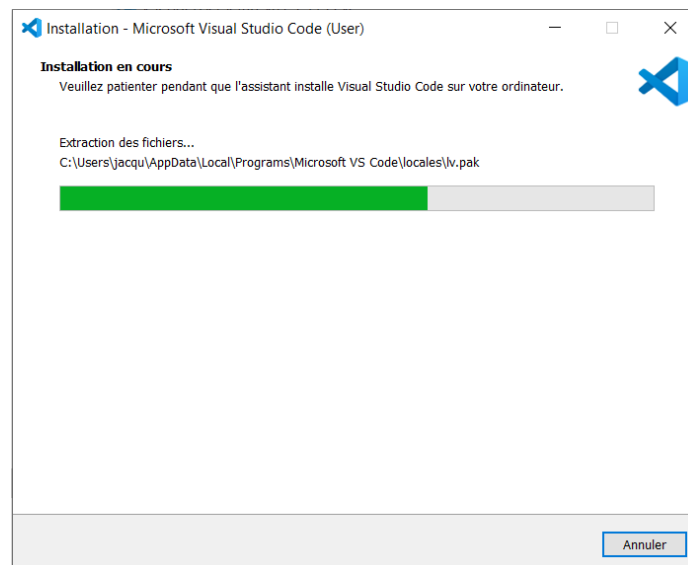
³³ On aurait pu obtenir le même résultat avec une page Web statique et un script client, mais comme l'objectif était de simuler un traitement des paramètres sur le serveur, on a préféré utiliser un script serveur et montrer au passage comment créer une page Web dynamique, comme en ASP ou PHP.

³⁴ Visual Studio Code : <https://code.visualstudio.com/>

Là encore, pour l'installer sur un PC, il suffit de télécharger :

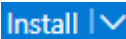
VSCodeUserSetup-x64-1.99.1.exe³⁵

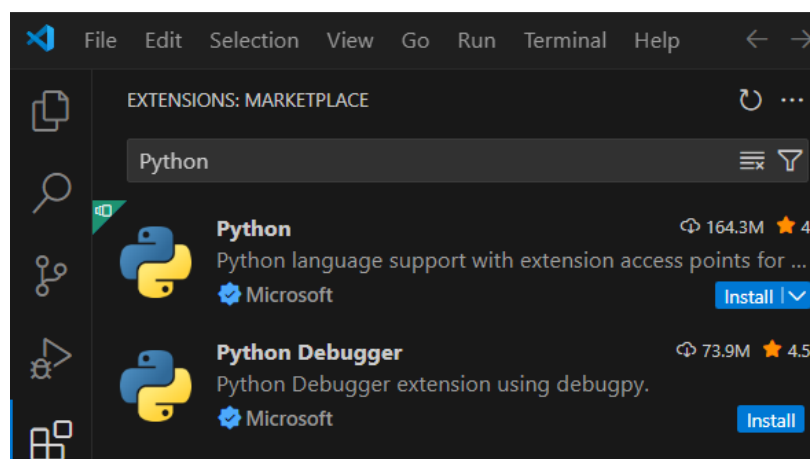
puis de l'exécuter en acceptant les options par défaut :



VS Code est « juste » une interface de programmation, adaptable à de nombreux langages et environnements de développements, mais qui offre de beaucoup de fonctionnalités très utiles : débogage, mise en évidence de la syntaxe, complétion intelligente du code, accès aux bibliothèques, etc. On peut le compléter par des extensions pour s'adapter à de multiples contextes.³⁶

Dans notre cas, il suffit de choisir l'extension **Python** de Microsoft pour programmer dans ce langage.

On peut l'installer en cliquant sur  puis sur 



Ceci installe en fait 3 extensions³⁷, **PyLance**, **Python** et **Python Debugger**, qui vont permettre d'utiliser très facilement le logiciel Python.

³⁵ Dernière version pour Windows sur <https://code.visualstudio.com/>

³⁶ https://fr.wikipedia.org/wiki/Visual_Studio_Code

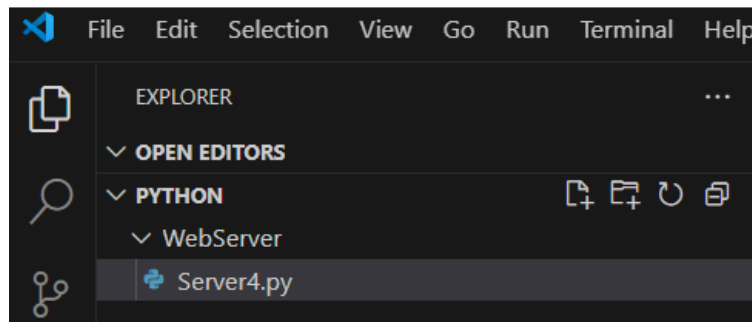
³⁷ On peut bien sûr en installer d'autres, en procédant de la même manière ; par exemple l'extension Open in Notepad++ qui permet de récupérer des codes bien formatés, en faisant appel à Notepad++ et son module d'extension NppExport. Ce dernier permet en effet de copier un texte RTF dans le presse-papiers : cette commande a été utilisée pour pouvoir coller dans Word le code du programme précédent.

Pour créer le programme Python considéré précédemment, on va d'abord créer dans le PC un dossier **Python** qui contiendra tous les projets en Python.

Dans VS Code, la commande **File/Open Folder** en choisissant ce dossier permet de créer un nouvel espace de travail³⁸, **PYTHON**, dans lequel on pourra regrouper tous les projets en Python ; on pourra



y accéder en cliquant sur , puis ajouter dedans un sous-dossier **WebServer** contenant un fichier **server4.py**, en utilisant les commandes contextuelles **New Folder** puis **New File** :

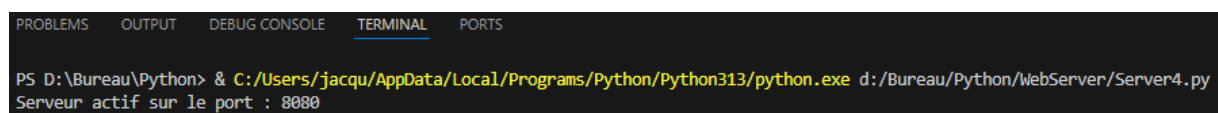


En sélectionnant **Server4.py**, on ouvre une fenêtre permettant d'éditer ce fichier, dans lequel on fera un copier-coller du code précédent :

```
Server4.py X
WebServer > Server4.py > ...
1 # server4.py
2 #
3 # Enable a Web server in IPv4 on the 8080 port which decodes 2 parameters
4 # in the URL and return their values in a dynamic HTML page.
5 # Call : http://<IPv4 PC address>:8080/?param1=<value1>&param2=<value2>
6
7 from http.server import BaseHTTPRequestHandler, HTTPServer
8 from urllib.parse import urlparse, parse_qs
9
10 serverPort = 8080
11
12 def printHTML(self, s):
13     self.wfile.write(bytes(s, "utf-8"))
14     self.wfile.write(bytes('\n', "utf-8"))
15
```



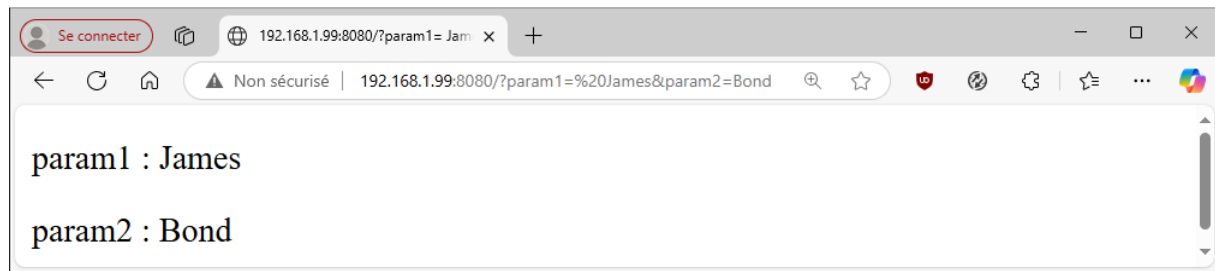
Pour exécuter ce programme, il suffit de cliquer sur  (Run Python File), en haut et à droite de cette fenêtre. Ceci ouvre une fenêtre **TERMINAL** qui ne mentionne aucune erreur et indique que le serveur est bien actif sur le port 8080 :



On peut vérifier le bon fonctionnement de ce server Web en l'appelant dans un navigateur Internet lancé sur ce même PC ; par exemple en utilisant Microsoft Edge, voici ce qu'on obtient avec l'URL suivante :

<http://192.168.1.99:8080/?param1=James¶m2=Bond>

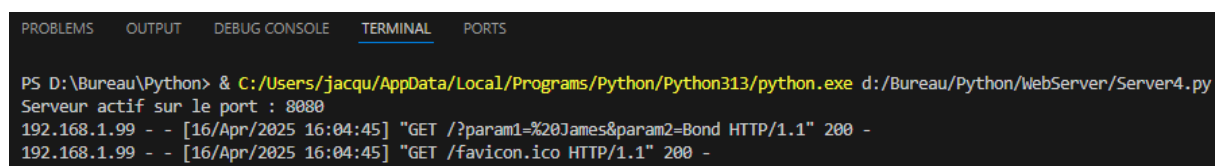
³⁸ Workspace dans VS Code



En cliquant sur **Afficher la source de la page** dans son menu contextuel (clic-D), on peut observer le code HTML de la page Web renvoyée par le serveur :



Côté serveur, la fenêtre **TERMINAL** affiche une trace des 2 commandes **HTTP GET** :



2.3.2 Server Web en IPv6

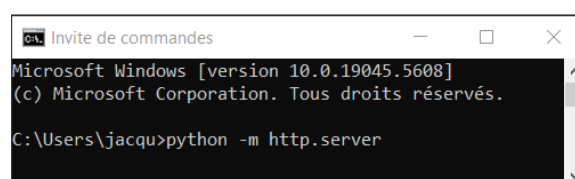
Même si l'on s'écarte un peu de l'objectif initial, il sera utile de savoir créer ici un serveur analogue au précédent, mais fonctionnant cette fois en **IPv6**, ceci pour bien montrer la possibilité d'accès distant en **IPv6**.

Pour implémenter un serveur Web en **IPv6**³⁹ qui réponde aux objectifs fixés, on va utiliser le programme Python suivant⁴⁰ :

```

# server6.py
#
# Enable a Web server in IPv6 on the 8080 port which decodes 2 parameters
# in the URL and return their values in a dynamic HTML page.
# Call : http://[<IPv6 PC address>]:8080/?param1=<value1>&param2=<value2>
  
```

³⁹ Là encore la commande suivante peut être utilisée :



mais sans pouvoir ici effectuer les traitements visés côté serveur.

⁴⁰ Adapté de <https://gist.github.com/akorobov/7903307>


```

import socket
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs

serverPort = 8080

def printHTML(self, s):
    self.wfile.write(bytes(s, "utf-8"))
    self.wfile.write(bytes('\n', "utf-8"))

class MyServer(BaseHTTPRequestHandler):
    def do_GET(self):
        # Decode parameters
        url_parts = urlparse(self.path)
        query_params = parse_qs(url_parts.query)
        val1 = query_params.get('param1', [''])[0]
        val2 = query_params.get('param2', [''])[0]

        # Response with dynamic web page
        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()
        printHTML(self, "<!DOCTYPE html>")
        printHTML(self, "<html>")
        printHTML(self, "    <body>")
        printHTML(self, "        <p>param1 : %s</p>" % val1)
        printHTML(self, "        <p>param2 : %s</p>" % val2)
        printHTML(self, "    </body>")
        printHTML(self, "</html>")

class HTTPServerV6(HTTPServer):
    address_family = socket.AF_INET6

webServer = HTTPServerV6(("", serverPort), MyServer)
print("Serveur actif sur le port :", serverPort)
webServer.serve_forever()

```

Comme le précédent, ce programme définit un script serveur, **do_GET**, qui répond à une requête **HTTP GET** de la forme :

http://[<adresse IPv6 du PC>]:8080/?param1=<valeur1>& param2=<valeur2>⁴¹

Il décode les valeurs des 2 paramètres et retourne une page Web dynamique affichant ces valeurs.

Pour cela, il définit une classe **HTTPServerV6** qui hérite de **HTTPServer**, mais dont le champ **address_family** est cette fois fixé à **socket.AF_INET6** (au lieu de **AF_INET**, valeur par défaut dans **HTTPServer**, pour **IPv4**) ; ceci permet de remplacer l’instruction :

```
webServer = HTTPServer(("", serverPort), MyServer)
```

par :

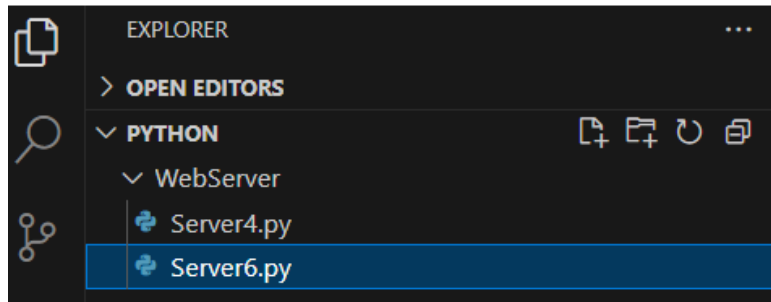
```
webServer = HTTPServerV6(("", serverPort), MyServer)
```

Elle indique que le serveur « écoute » cette fois les requêtes en IPv6.

On procédera ensuite comme précédemment pour exécuter ce programme avec VS Code.

- Ajout d’un fichier **Server6.py** dans le dossier WebServer :

⁴¹ On notera la présence de crochets autour de l’adresse IPv6, pour différencier les ‘:’ de cette adresse du ‘:’ précédant le port



- Edition de **Server6.py** et copier-coller du code précédent :

```

Server4.py  Server6.py
WebServer > Server6.py > ...
1  # server6.py
2  #
3  # Enable a Web server in IPv6 on the 8080 port which decodes 2 parameters
4  # in the URL and return their values in a dynamic HTML page.
5  # Call : http://[<IPv6 PC address>]:8080/?param1=<value1>& param2=<value2>
6
7  import socket
8  from http.server import BaseHTTPRequestHandler, HTTPServer
9  from urllib.parse import urlparse, parse_qs
10
11  serverPort = 8080
12
13  def printHTML(self, s):
14      self.wfile.write(bytes(s, "utf-8"))
15      self.wfile.write(bytes('\n', "utf-8"))
16

```

- Clic sur  (Run Python File), en haut et à droite de cette fenêtre :

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Bureau\Python> & C:/Users/jacqu/AppData/Local/Programs/Python/Python313/python.exe d:/Bureau/Python/WebServer/Server6.py
Serveur actif sur le port : 8080

```

- Appel du server en utilisant Microsoft Edge, avec l'URL suivante :

[http://\[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:7945\]:8080/?param1=James¶m2=Bond](http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:7945]:8080/?param1=James¶m2=Bond)



Le code source est identique :



Les logs diffèrent juste au niveau de l'adresse IP :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Bureau\Python> & C:/Users/jacqu/AppData/Local/Programs/Python/Python313/python.exe d:/Bureau/Python/WebServer/Server6.py
Serveur actif sur le port : 8080
2a02:7945: - - [16/Apr/2025 16:20:53] "GET /?param1=James&param2=Bond HTTP/1.1" 200 -
2a02:7945: - - [16/Apr/2025 16:20:53] "GET /favicon.ico HTTP/1.1" 200 -
```

Remarque importante

Précédemment, on a fait un accès local au serveur IPv6 qui tourne sur le port 8080 du PC. Mais pour qu'un accès distant à ce serveur puisse fonctionner, en utilisant l'adresse IPv6 du PC, il faudra autoriser l'accès dans la Box, en ajoutant la règle suivante dans son **pare-feu IPv6**, là encore en se connectant comme administrateur à son site Web (<http://192.168.1.1>) :

- Item de menu **Home > Réseau v6 > Pare-feu** :

Filtrage d'adresses						
#	Nom	IP source (Internet)	Protocole	Type	Ports	IP de destination (Lan)
1	Web_Server_PC	*	TCP	Port	8080	2a02:7945

2.4 Accès local et distant avec un Smartphone

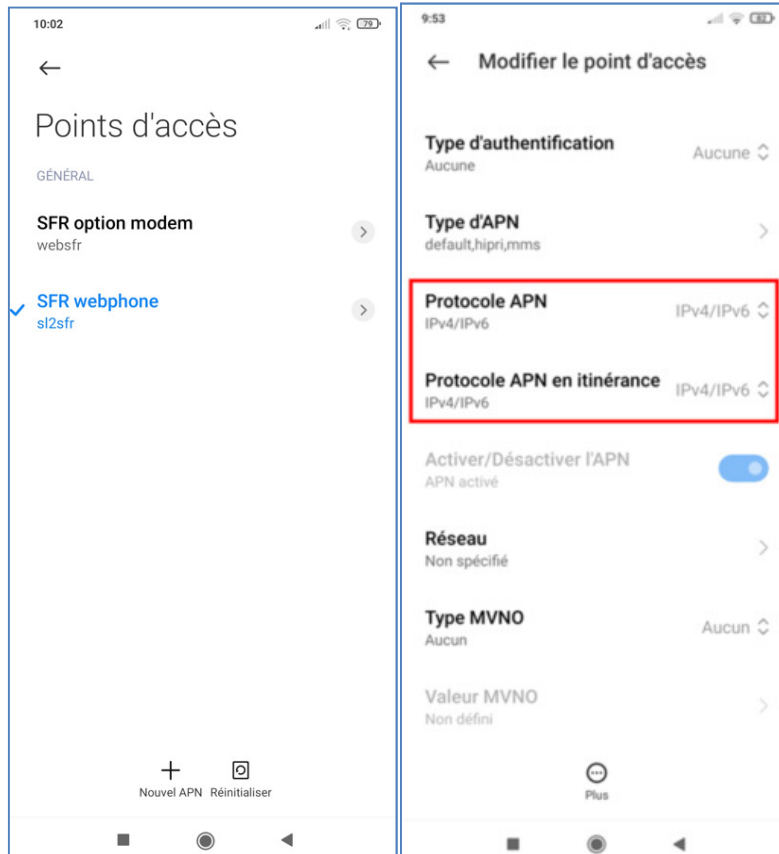
Un Smartphone peut être très facilement utilisé pour tester les accès à un serveur Web local.

- On le connectera en Wifi à la box pour tester un accès local.
- On supprimera cette connexion et on activera les données mobiles pour tester un accès distant effectif.

Auparavant, il faudra si nécessaire étendre le protocole IP utilisé lors d'un accès distant⁴², en modifiant ses paramètres **APN**⁴³. On peut le faire via l'application **Paramètres**, en affichant la liste des **Points d'accès** :

⁴² En accès local, comme pour le PC, les 2 protocoles IPv4 et IPv6 sont pris en charge ; on peut le constater avec **Paramètres/Wi-Fi/Paramètres supplémentaires/Adresse IP**, équivalent ici de la commande **IPConfig**.

⁴³ Access Point Name.



Si on utilise le navigateur Firefox, sur un Smartphone Xiaomi MI9 SE fonctionnant sous Android 10, avec l'une des 2 URL suivantes dans un navigateur Internet :

- IPv4 : <http://192.168.1.99:8080/?param1=James¶m2=Bond>.
- IPv6 : [http://\[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:7945\]:8080/?param1=James¶m2=Bond](http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:7945]:8080/?param1=James¶m2=Bond).

voici ce qu'on obtient comme page Web quand on appelle un des serveurs précédents sur le PC :

- Serveur IPv4

Smartphone	IPv4	IPv6
Accès local	param1 : James param2 : Bond	KO
Accès distant		KO

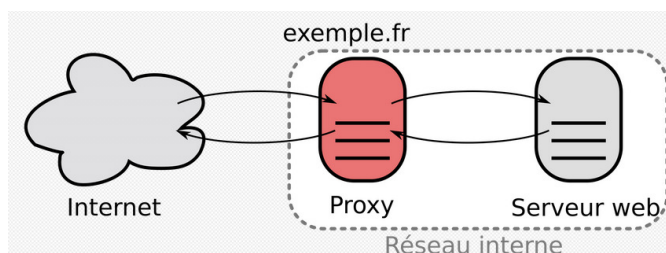
- Serveur IPv6

Smartphone	IPv4	IPv6
Accès local	KO	param1 : James param2 : Bond
Accès distant		param1 : James param2 : Bond

On notera ici que le problème à résoudre est celui de la case jaune du premier tableau précédent.

2.5 Proxy Inverse et NGINX

Un proxy inverse est un serveur qui « permet à un utilisateur d'Internet d'accéder à des serveurs internes »⁴⁴, c'est-à-dire situés dans un réseau privé :



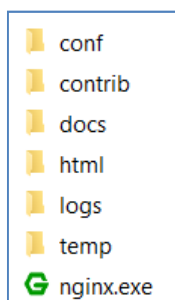
Il est donc accessible publiquement et il sera souvent utilisé pour :

- Contrôler les accès.
- Optimiser le trafic grâce à des mises en cache, des compressions, des équilibrages de charges...
- Sécuriser les échanges (SSL...).

Le serveur **NGINX**^{45 46}, créé en 2002 par Igor Sysoev sous Linux et qui a fait depuis l'objet de très nombreux développements est maintenant l'un des plus utilisés au monde pour les raisons suivantes :

- Open source sous licence BSD⁴⁷.
- Multiples fonctionnalités : serveur Web, serveur proxy direct et inverse, serveur de streaming, etc.
- Fonctionne sous GNU/Linux, BSD, Mac OS X, Solaris, Microsoft Windows, etc.

Sous Windows, et toujours sur le PC considéré précédemment, on peut très facilement découvrir comment l'utiliser en téléchargeant gratuitement et en décompressant l'archive **nginx-1.26.2.zip**⁴⁸ dans un dossier accessible en lecture et écriture ; ceci crée l'arborescence suivante :



Il suffit d'exécuter en tant qu'administrateur le fichier **nginx.exe** pour le lancer le serveur, après avoir modifié son fichier de configuration, **nginx.conf**, situé dans le sous-dossier **conf**.

Pour le lancer comme proxy inverse, il suffit par exemple d'utiliser un fichier de configuration tel que⁴⁹ :

⁴⁴ https://fr.wikipedia.org/wiki/Proxy_inverse

⁴⁵ <https://fr.wikipedia.org/wiki/NGINX>

⁴⁶ <https://nginx.org/en/>

⁴⁷ https://fr.wikipedia.org/wiki/Licence_BSD

⁴⁸ <https://nginx.org/en/download.html>

⁴⁹ <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>.

```
# nginx.conf

worker_processes 1;

events
{
    worker_connections 1024;
}

http
{
    include mime.types;
    default_type application/octet-stream;
    sendfile on;
    keepalive_timeout 65;
    server
    {
        listen [::]:80;

        location /
        {
            proxy_pass http://192.168.1.99:8080/;
        }

        # redirect server error pages to the static page /50x.html
        error_page 500 502 503 504 /50x.html;
        location = /50x.html
        {
            root html;
        }
    }
}
```

Ceci est simple à comprendre :

- L'instruction `listen[::]:80` indique que le serveur **NGINX** va répondre à des requête HTTP en IPv6 sur le port 80⁵⁰.
- Les instructions clés :

```
location /
{
    proxy_pass http://192.168.1.99:8080/;
}
```

précisent la redirection à effectuer IPv6:80 => IPv4:8080, vers le serveur Web local, en utilisant l'adresse IPv6 du PC et éventuellement des paramètres⁵¹.

Comme l'accès externe en IPv6 se fera cette fois sur le port 80, il faudra modifier en conséquence la règle précédemment ajoutée dans le pare-feu de la box :

- Item de menu **Home > Réseau v6 > Pare-feu** :

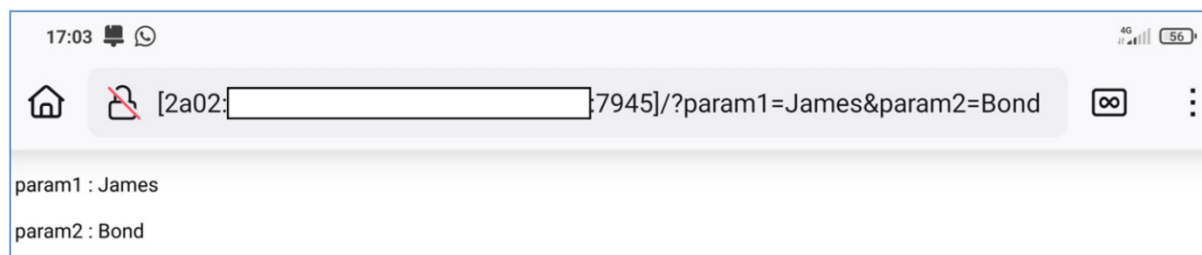
Filtrage d'adresses						
#	Nom	IP source (Internet)	Protocole	Type	Ports	IP de destination (Lan)
1	Nginx_PC	*	TCP	Port	80	2a02: 7945

⁵⁰ On peut l'omettre car c'est ici le port par défaut.

⁵¹ L'utilisation éventuelle de paramètres est indiquée par le /, à la fin de l'instruction `proxy_pass`.

Dès lors, en lançant à la fois NGINX et le serveur Web IPv4 du §2.3.1 sur le PC, on peut accéder à distance à ce dernier, en IPv6, à partir du Smartphone n'utilisant que des données mobiles et avec l'URL suivante⁵² :

[http://\[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:7945\]/?param1=James¶m2=Bond](http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:7945]/?param1=James¶m2=Bond)



Le proxy inverse **NGINX** a donc bien permis de faire ici la conversion IPv6 => IPv4 que nous n'avons pas réussi à obtenir de la box.

3 Solution opérationnelle

3.1 Routeur GL-MT300N-V2

Si l'expérimentation précédente a le mérite de la simplicité pour découvrir NGINX, elle n'est pas très intéressante pour résoudre le problème initial d'accès aux serveurs de notre installation photovoltaïque car il n'est pas pertinent de devoir faire fonctionner en permanence un PC pour autoriser un accès distant à ces derniers.

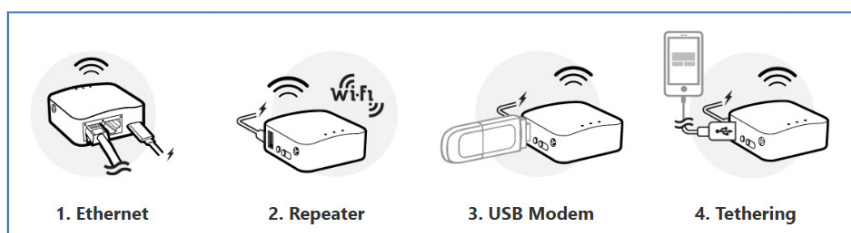
3.1.1 Présentation du routeur

Une solution moins coûteuse a donc été recherchée pour faire fonctionner NGINX et le routeur **GL-MT300N-V2**⁵³ de **GL.iNet** en est une, particulièrement intéressante :



pour les raisons suivantes :

- On le trouve à moins de 25€ sur Internet.
- Alimentation par prise micro USB de type C (5V, 2A max).
- Routeur Wifi connectable à Internet par différents moyens :



⁵² On peut omettre la mention du port car 80 est le port par défaut pour une requête HTTP.

⁵³ <https://www.gl-inet.com/products/gl-mt300n-v2/>

- 300Mb/s max en Wifi.
- Facilement paramétrable.
- VPN.
- Nombreux tutoriaux.
- **OpenWrt avec NGINX.**
- **IPv4 et IPv6.**
- Mémoire (DDR2 128MB/NOR Flash 16MB) extensible avec une clé USB.
- Bouton Reset : appuyé 4s => reboot et 10s min => reset usine.
- Etc.

Ce petit routeur « de voyage » est donc un véritable « couteau Suisse » !⁵⁴

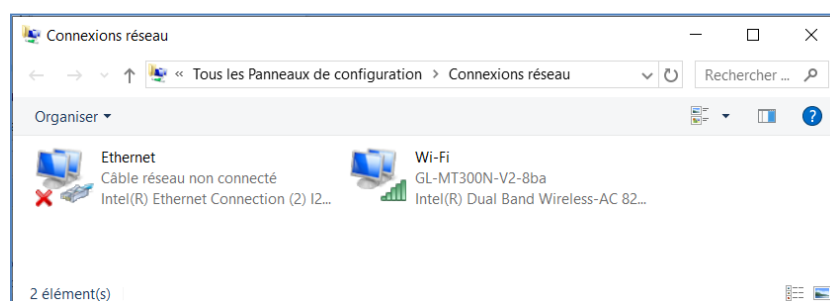
Quand on l'alimente via sa prise micro USB de type C, il active⁵⁵ un point d'accès Wifi pour lequel :

- SID = **GL-MT300N-V2-xxx**
- PWD = **goodlife**

Dans tout ce qui suit, on va faire fonctionner le routeur en mode **Ethernet**, en le connectant à la box via un câble réseau sur sa prise RJ45 WLAN.

3.1.2 Désactivation d'IPv6 sur le PC

Pour le tester, on va utiliser le PC précédemment considéré, mais en restreignant son fonctionnement en IPv4, car notre objectif est de montrer comment accéder à distance à son serveur IPv4⁵⁶. Ceci peut se faire avec **PARAMETRES / Réseau et Internet / Etat / Modifier les options de l'adaptateur** :

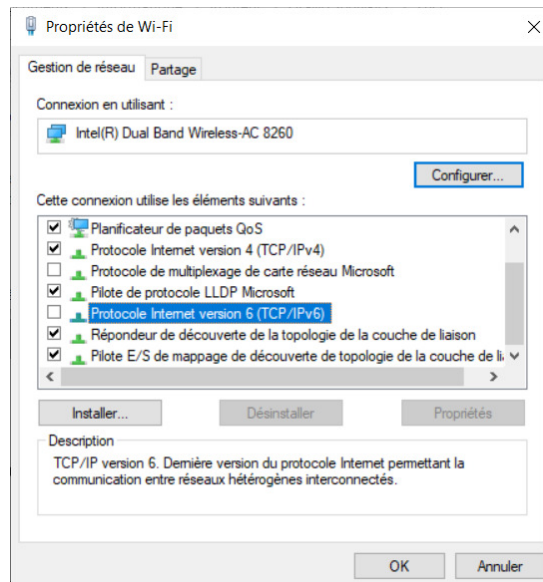


Dans le menu contextuel de l'adaptateur Wifi (Clic-D), il suffit de décocher la case Protocole **Internet version 6 (TCP/IPv6)** :

⁵⁴ A cause de sa couleur, on l'appelle parfois « **routeur Mango** ».

⁵⁵ Attendre que les 3 leds soient allumées (>1mn).

⁵⁶ Comme on l'a dit, le serveur IPv6 sur le PC n'a été proposé que pour mon montrer qu'on pouvait y accéder à distance en IPv6 et le protocole IPv6 sur le PC nous a servi pour tester Nginx en IPv6 sur le PC.



En tapant **IPCONFIG** dans l'invite de commandes, on peut constater que le PC ne travaille plus dès lors qu'en IPv4 :

```
Carte réseau sans fil Wi-Fi :
    Suffixe DNS propre à la connexion. . . . : 
    Adresse IPv4. . . . . : 192.168.1.99
    Masque de sous-réseau. . . . . : 255.255.255.0
    Passerelle par défaut. . . . . : 192.168.1.1
```

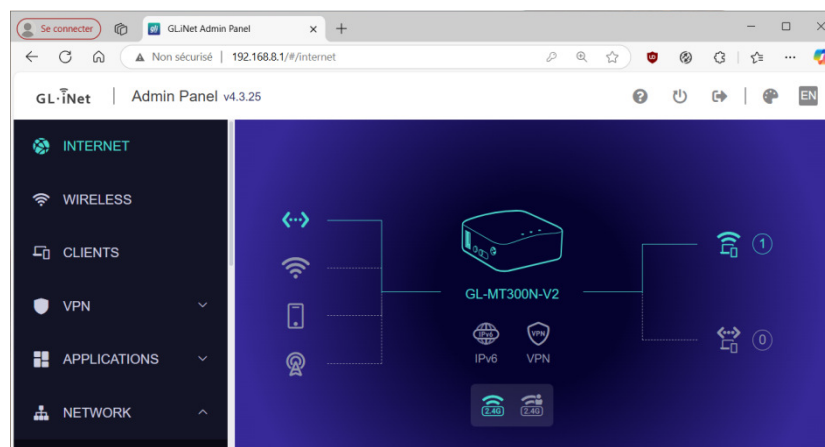
3.1.3 Panneau d'administration du routeur

Dès lors, après avoir connecté le PC en Wifi cette fois au routeur (et non à la box), on peut ouvrir son **Panneau d'administration**⁵⁷ avec l'URL :

<http://192.168.8.1>

A la première connexion⁵⁸, on doit choisir un langage (English ici), définir un mot de passe pour administrer le routeur (xxxxxxxxx ici) et lancer éventuellement une mise à jour du système⁵⁹.

Puis on affiche une page correspondant à l'item de menu **INTERNET** :



⁵⁷ <https://images-na.ssl-images-amazon.com/images/I/B1oCL2RA8YS.pdf>

⁵⁸ https://docs.gl-inet.com/router/en/3/setup/mini_router/first_time_setup

⁵⁹ Version v4.3.25 actuellement.

Par défaut, le routeur fonctionne en **mode Router** et en **IPv4**, c'est-à-dire qu'il va créer et gérer un sous-réseau privé IPv4 avec comme adressage 192.168.8.x.

En vue de le rendre accessible à distance en IPv6, on va commencer par activer également le protocole IPv6 sur le routeur, avec l'item de menu **NETWORK > IPv6** :

Enabled IPv6 

Dès lors, la box lui attribuera à la fois une adresse privée IPv4 et une adresse publique IPv6. Après avoir reconnecté en Wifi le PC au routeur et rouvert son Panneau d'administration, voici ce qu'indique maintenant l'item de menu **INTERNET** :

Ethernet

Protocol

DHCP

IPv4

IPv6

IP Address

192.168.1.34

IPv4

IPv6

IP Address

2a02: :beba/128

Les services DHCP du box lui ont donc attribué à la fois une adresse IPv4 et une adresse IPv6. Là encore on gagnera à les rendre statiques, avec les items de menu **Réseau v4 > DHCP** et **Réseau v6 > DHCP** de la box, en s'y connectant (<http://192.168.1.1>) comme Administrateur :

Adresses statiques

Adresse IP

Adresse MAC

192.168.1.34

94: :ba

Adresse IP

Adresse MAC

2a02: :beba

94: ba

Et le routeur agira de même dans son sous-réseau, en attribuant à chacun de ses postes des adresses privées IPv4 et des adresses publiques IPv6 à ceux qui sont capables de les gérer, ce qui n'est plus le cas maintenant pour le PC. On peut le constater, en tapant **IPCONFIG** dans son invite de commandes (cmd) :

```

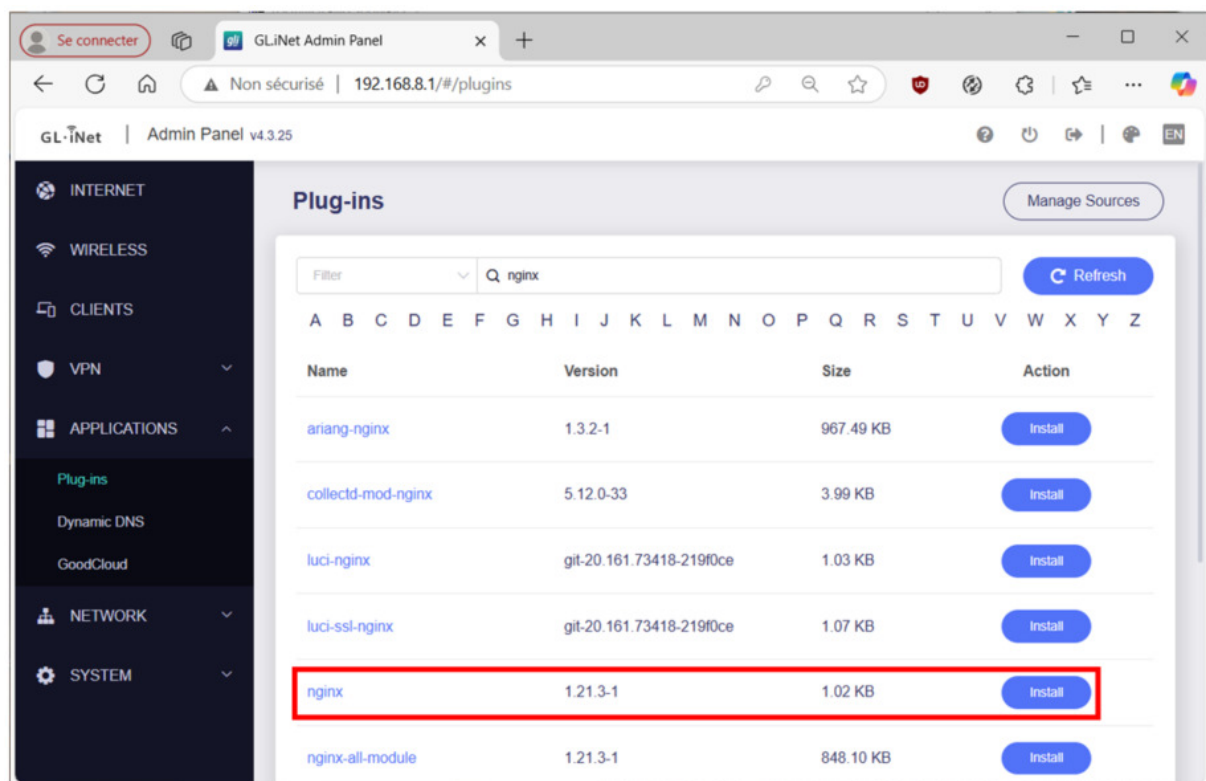
Carte réseau sans fil Wi-Fi :

Suffixe DNS propre à la connexion. . . : lan
Adresse IPv4. . . . . : 192.168.8.237
Masque de sous-réseau. . . . . : 255.255.255.0
Passerelle par défaut. . . . . : 192.168.8.1

```

Là encore on rendra **statique** cette adresse IPv4, 192.168.8.237, mais en paramétrant cette fois le service DHCP IPv4 du routeur, à l'aide d'un autre outil d'administration de ce dernier : LuCI.

Mais avant de le faire, on peut constater l'intérêt d'un tel routeur en affichant la liste des nombreuses applications installables⁶⁰ avec l'item **APPLICATIONS > Plug-ins** ; elle contient en particulier **NGINX** :



Souvent, pour installer une nouvelle application, il faut augmenter la mémoire flash du routeur, en utilisant par exemple une clé USB (cf. §5.3). Mais ça ne sera pas nécessaire ici car on va voir NGINX est installé par défaut sur le routeur !

3.1.4 LuCI

Pour aller plus loin il faut utiliser un second outil d'administration du routeur, **LuCI**⁶¹, que l'on peut lancer ici avec l'item de menu **SYSTEM > Advanced Settings** :

You can modify advanced settings with LuCI, the default web user interface of OpenWrt. LuCI is an open and independent project maintained by OpenWrt.

It is provided as is. GL.iNet is not responsible for LuCI maintenance.

192.168.8.1/cgi-bin/luci

⁶⁰

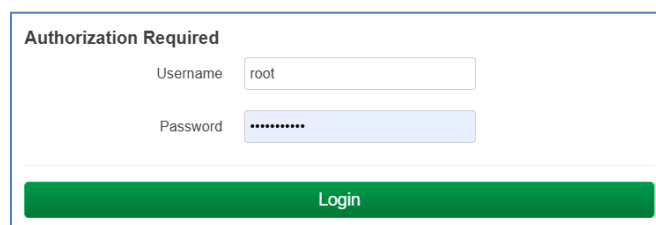
⁶¹ Lua Control Interface.

ou directement avec l'URL précédente :

<http://192.168.8.1/cgi-bin/luci>

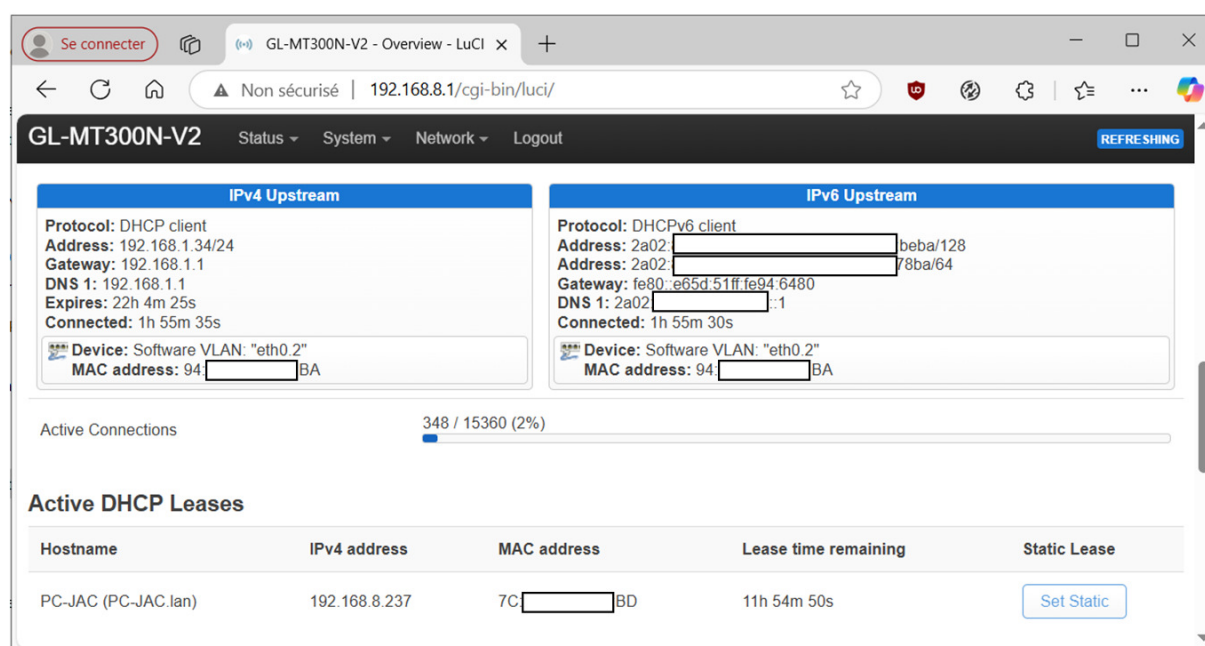
Ceci active l'interface graphique du système **OpenWrt**⁶². C'est un système ouvert, distribué sous licence GNU, basé sur Linux et utilisable comme système de base dans les routeurs ; et c'est bien le cas pour notre routeur.

Quand on le lance, il faut s'identifier comme **root**, avec le même mot de passe que celui précédemment défini pour le routeur (xxxxxxxxx donc) :



Là encore on dispose d'une arborescence de menus pour superviser et paramétrer le routeur.

Par exemple, l'item de menu **Status > Overview** affiche en particulier l'état du réseau :



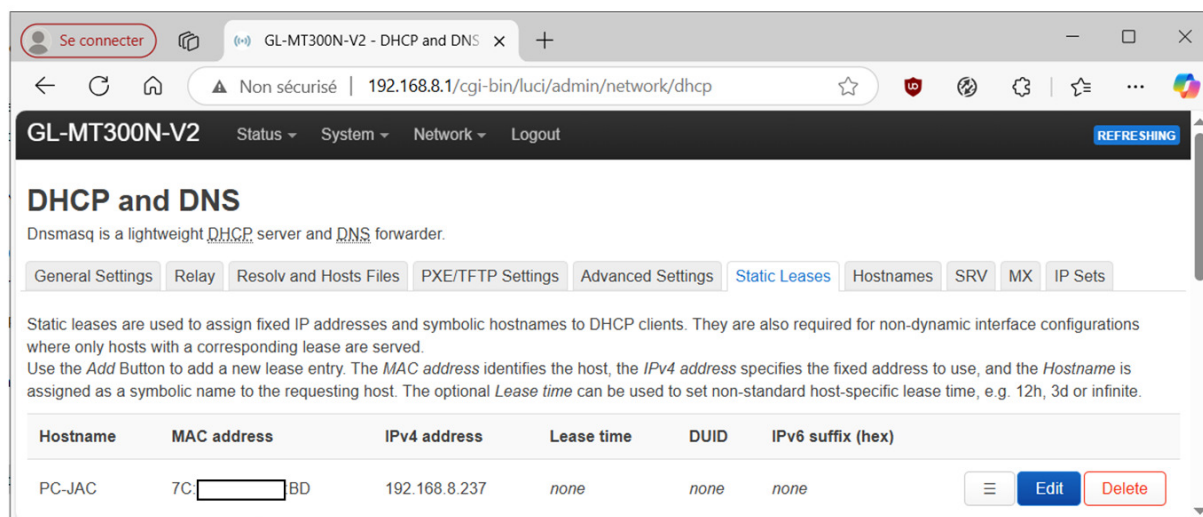
Hostname	IPv4 address	MAC address	Lease time remaining	Static Lease
PC-JAC (PC-JAC.lan)	192.168.8.237	7C:BD	11h 54m 50s	<button>Set Static</button>

Le routeur fonctionne donc bien en IPv4 et IPv6 et on retrouve ici les adresses IP attribuées au routeur par les services DHCP de la box.

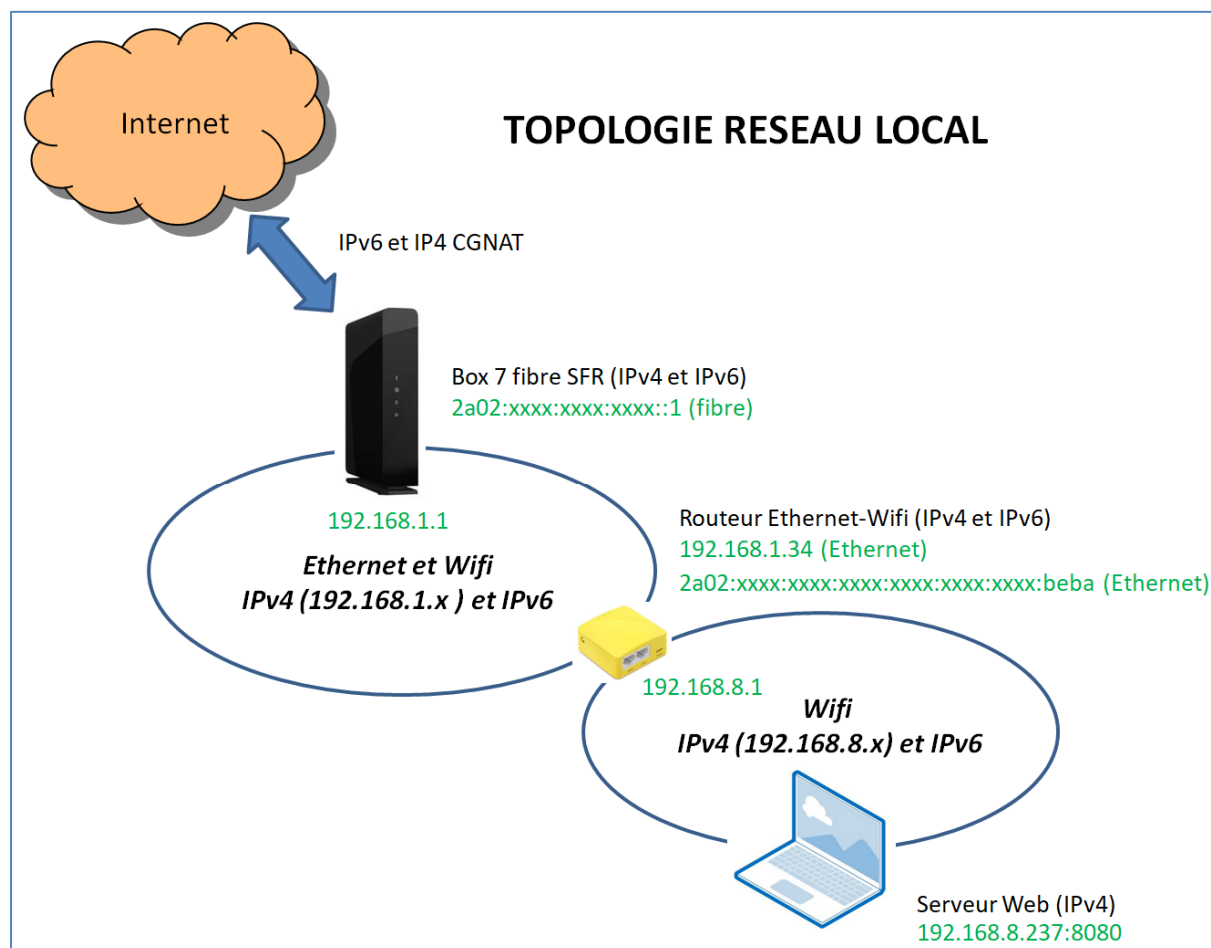
Il permet également de voir les postes connectés dans le sous-réseau privé créé par le routeur, ainsi que les adresses IPv4 et IPv6 attribuées par les services DHCP du routeur cette fois ; dans notre cas, le seul poste connecté est le PC.

En cliquant sur le bouton Set Static, on peut rendre statique son adresse IPv4 et le constater avec l'item de menu **Network > DHCP and DNS > Static Leases** :

⁶² <https://openwrt.org/docs/start>.



En résumé, voici la topologie du réseau maintenant considéré, dont toutes les adresses IP sont statiques :



LuCI permet aussi de paramétrer finement le pare-feu du routeur. Par exemple, on peut constater que l'accès distant au Panneau d'administration ou à LuCI, à partir du Smartphone en IPv6 ne fonctionne pas. Pour le faire fonctionner, il suffit d'ajouter la règle suivante de trafic, en utilisant l'item de menu **Network > Firewall > Traffic Rules** :

Firewall - Traffic Rules

Traffic rules define policies for packets traveling between different zones, for example to reject traffic between certain hosts or to open WAN ports on the router.

Traffic Rules

Name	Match	Action	Enable
Admin	Incoming <i>IPv4 and IPv6</i> From <i>any zone</i> To <i>this device</i> , port <i>80</i>	Accept input	☒

Elle est définie par les paramètres suivants :

Firewall - Traffic Rules - Admin

[General Settings](#) [Advanced Settings](#) [Time Restrictions](#)

Name

Admin

Protocol

TCP | UDP

Source zone

Any zone (forward)

Source address

-- add IP --

Source port

any

Destination zone

Device (input)

Destination address

-- add IP --

Destination port

80

Action

accept

Il faut cliquer sur [Save & Apply](#) pour l'ajouter à la liste des règles existantes.

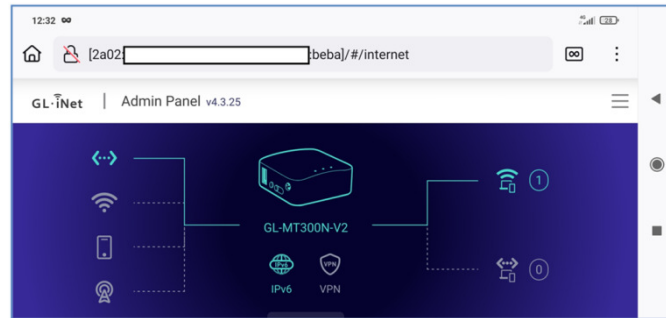
Evidemment, il faut aussi créer dans le pare-feu de la box une règle autorisant l'accès en IPv6 au site Web du routeur, comme on l'avait fait pour le PC. On utilisera l'item de menu **Home > Réseau v6 > Pare-feu**, on pourra remplacer cette dernière par :

Filtrage d'adresses						
#	Nom	IP source (Internet)	Protocole	Type	Ports	IP de destination (Lan)
1	Routeur	*	TCP	Port	80	2a02: beba

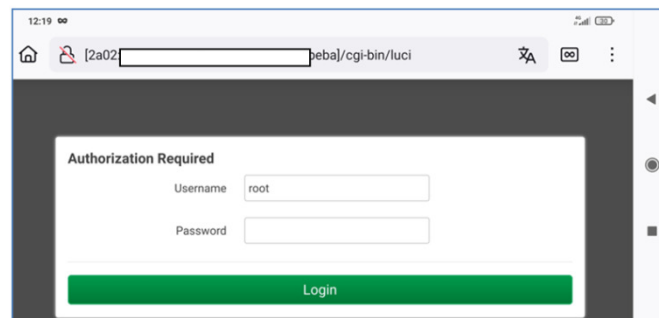
en utilisant cette fois l'adresse IPv6 du routeur.

Dès lors, voici par exemple ce qu'on obtient sur le Smartphone, fonctionnant en mode données mobiles, avec des URL basées sur l'adresse IPv6 du routeur :

[http://\[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:beba\]](http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:beba])



[http://\[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:beba\]/cgi-bin/luci](http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:beba]/cgi-bin/luci)

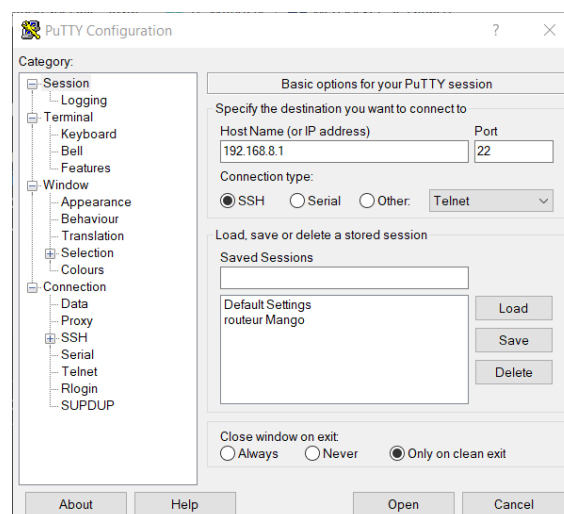


On peut donc bien ainsi administrer à distance le routeur, en formulant des requêtes HTTP IPv6 à son site Web, sur le port 80.

3.1.5 Commandes SSH et Putty

Pour aller plus loin, on va utiliser une troisième possibilité d'administration du système OpenWrt sur le routeur : formuler des **commandes Linux en SSH**⁶³. On peut le faire avec **Putty**⁶⁴, après l'avoir installé en téléchargeant **putty-64bit-0.83-installer.msi**⁶⁵ pour le PC sous Windows.

Quand on le lance sur le PC, il suffit d'indiquer l'adresse IPv4 du routeur (192.168.8.1) et de choisir le mode SSH⁶⁶ :



⁶³ Secure SHell

⁶⁴ <https://www.putty.org/>.

⁶⁵ <https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>.

⁶⁶ On peut sauvegarder ces paramètres de session, ce qu'on a fait ici sous le nom « **routeur Mango** ».

En cliquant sur **Open**, on ouvre un éditeur de commandes dans lequel il faut commencer par s'identifier :

- login as : **root**
- root @192.168.8.1's password : **Pierrefeu06**⁶⁷

```
root@GL-MT300N-V2: ~
login as: root
root@192.168.8.1's password:

BusyBox v1.35.0 (2023-04-09 12:27:46 UTC) built-in shell (ash)

  _   _   _   _   _   _   _   _   _   _   _   _   _   _   _   _
 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
 | |_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|
  |_| W I R E L E S S   F R E E D O M

-----
OpenWrt 22.03.4, r20123-38ccc47687
-----
root@GL-MT300N-V2:~#
```

Avec cet outil, on peut vite constater que NGINX a été lancé sur le routeur avec la commande **service** :

```
root@GL-MT300N-V2:~# service
Usage: service <service> [command]

/etc/init.d/boot          enabled      stopped
/etc/init.d/bootcount     enabled      stopped
/etc/init.d/carrier-monitor enabled      running
/etc/init.d/cron          enabled      stopped
/etc/init.d/dnscrypt-proxy enabled      stopped

...

/etc/init.d/mwan3         enabled      stopped
/etc/init.d/network       enabled      running
/etc/init.d/nginx         enabled      running
/etc/init.d/odhcpd        enabled      running
/etc/init.d/openssl       enabled      stopped

...

/etc/init.d/usbmuxd       enabled      stopped
/etc/init.d/vpnpolicy     enabled      stopped
/etc/init.d/vpnpolicy-apply enabled      stopped
root@GL-MT300N-V2:~#
```

qui précise également son script de lancement : **/etc/init.d/nginx**.

Si on arrête le service NGINX avec la commande **service nginx stop**, on peut constater que ceci désactive le Panneau de d'administration et LuCI. Et dès qu'on le relance avec la commande **service nginx start**, ceux-ci sont réactivés. Ainsi NGINX est déjà utilisé ici comme serveur Web.

Et on a vu que ce serveur écoutait bien en IPv6 sur le port 80 (cf. §3.1.4).

Peut-on dès lors utiliser ici également NGINX comme proxy inverse ? C'est ce que nous allons essayer de faire dans le paragraphe suivant.

⁶⁷ Le même que celui défini pour le Panneau d'Administration.

3.2 Ajout de la fonctionnalité Proxy inverse à NGINX

Pour y parvenir, il suffit juste de comprendre l'enchaînement de fichiers de configuration de NGINX dans le routeur.

D'après le §3.1.5, le service NGINX est lancé par le script `/etc/init.d/nginx`.

Avec **Putty**, en entrant les commandes :

- `cd /etc/init.d`
- `cat nginx`

on peut afficher ce script :

```
#!/bin/sh /etc/rc.common
# Copyright (C) 2015 OpenWrt.org

START=80

USE_PROCD=1

start_service() {
    [ -f /etc/init.d/uhttpd ] && {
        /etc/init.d/uhttpd enabled && {
            /etc/init.d/uhttpd stop
            /etc/init.d/uhttpd disable
        }
    }

    [ -d /var/log/nginx ] || mkdir -p /var/log/nginx
    [ -d /var/lib/nginx ] || mkdir -p /var/lib/nginx

    procd_open_instance
    procd_set_param command /usr/sbin/nginx -c /etc/nginx/nginx.conf -g 'daemon off;'
    procd_set_param file /etc/nginx/nginx.conf
    procd_set_param respawn
    procd_close_instance
}
```

Après avoir désactivé le service **uhttpd**⁶⁸ s'il fonctionne, il lance NGINX en exécutant `/usr/sbin/nginx` avec `/etc/nginx/nginx.conf` comme fichier de configuration.

Toujours avec **Putty**, les commandes :

- `cd /etc/nginx`
- `cat nginx.conf`

permettent de l'afficher :

```
user root;
worker_processes 2;

pid /var/run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include mime.types;
    default_type application/octet-stream;

    sendfile on;
    keepalive_timeout 5;
```

⁶⁸ C'est habituellement le service qui lance LuCI dans OpenWrt.

```

    client_body_buffer_size 10K;
    client_header_buffer_size 1k;
    client_max_body_size 1G;
    large_client_header_buffers 2 2k;

    gzip_static on;

    root /www;

    access_log off;

    include /etc/nginx/conf.d/*.conf;
}

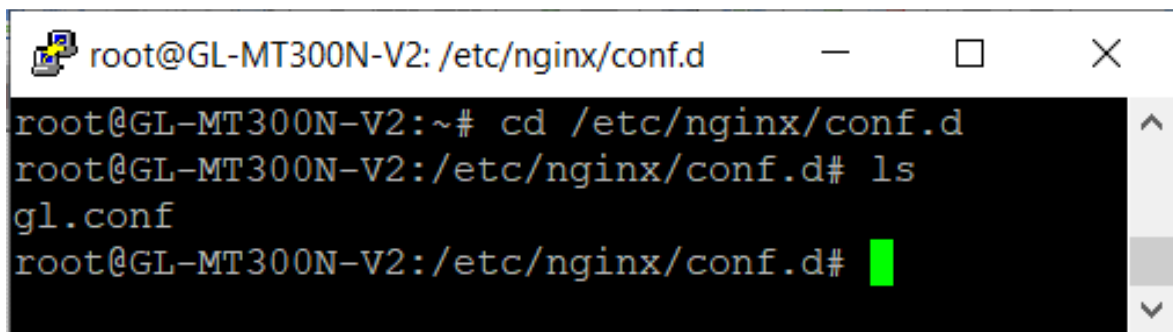
```

Dans ce fichier, la dernière instruction inclut tous les fichiers de configuration (*.conf) contenus dans **/etc/nginx/conf.d**.

Les commandes suivantes dans **Putty** :

- **cd /etc/nginx/conf.d**
- **ls**

permettent de constater qu'il n'y en a qu'un seul : **gl.conf** :



```

root@GL-MT300N-V2: /etc/nginx/conf.d
root@GL-MT300N-V2:~# cd /etc/nginx/conf.d
root@GL-MT300N-V2:/etc/nginx/conf.d# ls
gl.conf
root@GL-MT300N-V2:/etc/nginx/conf.d#

```

Là encore, on peut afficher son contenu avec :

- **cat gl.conf**

```

index gl_home.html;

lua_shared_dict shmem 12k;
lua_shared_dict nonces 16k;
lua_shared_dict sessions 16k;
lua_code_cache off;

init_by_lua_file /usr/share/gl-ngx/oui-init.lua;

server {
    listen 80;
    listen [::]:80;

    listen 443 ssl;
    listen [::]:443 ssl;

    ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
    ssl_prefer_server_ciphers on;
    ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+ECDSA+SHA256:EECDH+aRSA+SHA384:EECDH+aRSA+SHA256:EECDH:DHE+AESGCM:DHE:!RSA:!aNULL:!eNULL:!LOW:!RC4:!3DES:!MD5:!EXP:!PSK:!SRP:!DSS:!CAMELLIA:!SEED";
    ssl_session_tickets off;

    ssl_certificate /etc/nginx/nginx.cer;
    ssl_certificate_key /etc/nginx/nginx.key;

    resolver 127.0.0.1 ipv6=off;

    rewrite ^/index.html / permanent;
}

```

```

        location = /rpc {
            content_by_lua_file /usr/share/gl-ngx/ou
i-rpc.lua;
            add_header Content-Type application/json
;
        }

        location = /upload {
            content_by_lua_file /usr/share/gl-ngx/ou
i-upload.lua;
        }

        location = /download {
            content_by_lua_file /usr/share/gl-ngx/ou
i-download.lua;
        }

        location /cgi-bin/ {
            include fastcgi_params;
            fastcgi_read_timeout 300;
            fastcgi_pass unix:/var/run/fcgiwrap.sock
et;
        }

        location ~.*\.(html|png|jpg|svg)$ {
            add_header Cache-Control "private, no-st
ore, no-cache, must-revalidate, proxy-revalidate
";
        }

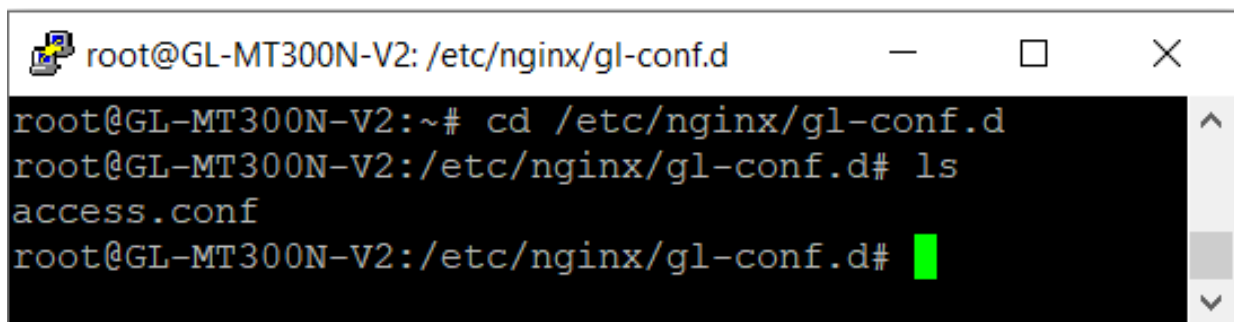
        include /etc/nginx/gl-conf.d/*.conf;
    }
}

```

Il configure NGINX comme un serveur Web qui écoute en IPv4 et IPv6 sur le port 80, mais pas comme un proxy inverse. Là encore la dernière instruction inclut tous les fichiers de configuration (*.conf) contenus dans **/etc/nginx/gl-conf.d**.

Comme précédemment, on peut les lister avec les commandes :

- **cd /etc/nginx/gl-conf.d**
- **ls**



```

root@GL-MT300N-V2: /etc/nginx/gl-conf.d
root@GL-MT300N-V2:~# cd /etc/nginx/gl-conf.d
root@GL-MT300N-V2:/etc/nginx/gl-conf.d# ls
access.conf
root@GL-MT300N-V2:/etc/nginx/gl-conf.d#

```

On constate qu'il n'y en a qu'un seul, **access.conf**, dont on peut là encore afficher le contenu avec :

- **cat access.conf**

```

location / {
    access_by_lua_file /usr/share/gl-ngx/oui-access.
    lua;
}

```

Là encore point de proxy inverse !

Mais cette technique d'inclusion de fichiers de configuration suggère une solution très simple pour ajouter la fonctionnalité de proxy inverse vers le serveur Web du PC :

Ajouter dans **/etc/nginx/gi-conf.d** un fichier de configuration, nommé **proxinv.conf** par exemple, contenant les instructions clés suivantes⁶⁹ :

```
location /websrv
{
    proxy_pass http://192.168.8.237:8080/;
}
```

Ainsi, on pourra appeler à distance ce serveur web avec une URL de la forme :

[http://\[adresse IPv6 du routeur\]/websrv?param1=<valeur1>¶m2=<valeur2>](http://[adresse IPv6 du routeur]/websrv?param1=<valeur1>¶m2=<valeur2>)

Là encore **Putty** peut être utilisé avec les instructions suivantes :

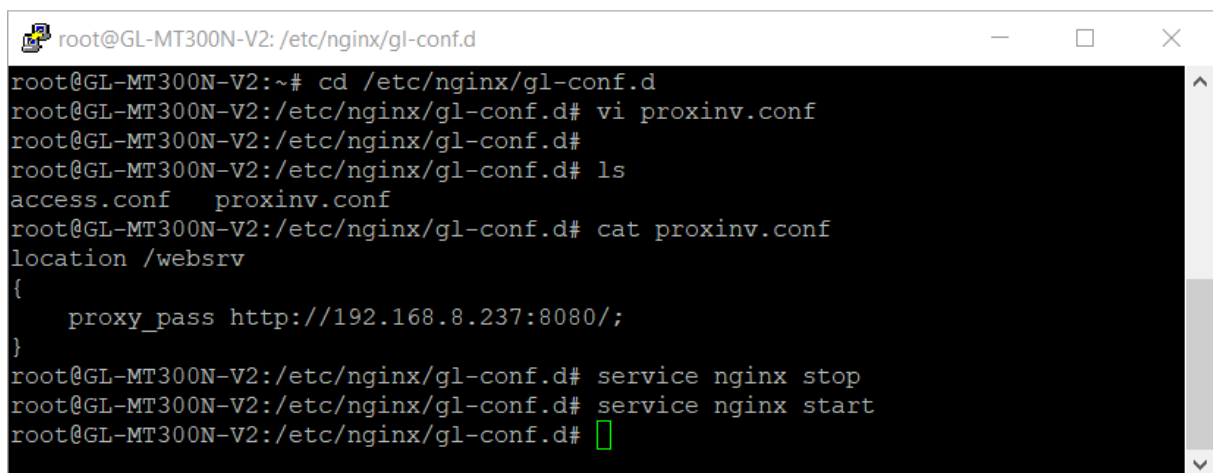
- **cd /etc/nginx/gi-conf.d**
- **vi proxinv.conf**

Dans **vi**, on entre :

- **i**
- **les instructions précédentes**⁷⁰
- **esc (touche)**
- **:w**
- **:q**

Et de retour dans **Putty**, pour vérifier puis prendre en compte ce nouveau fichier de configuration dans le service **nginx** :

- **ls**
- **cat proxinv.conf**
- **service nginx stop**
- **service nginx start**



```
root@GL-MT300N-V2: /etc/nginx/gi-conf.d
root@GL-MT300N-V2:~# cd /etc/nginx/gi-conf.d
root@GL-MT300N-V2:/etc/nginx/gi-conf.d# vi proxinv.conf
root@GL-MT300N-V2:/etc/nginx/gi-conf.d#
root@GL-MT300N-V2:/etc/nginx/gi-conf.d# ls
access.conf  proxinv.conf
root@GL-MT300N-V2:/etc/nginx/gi-conf.d# cat proxinv.conf
location /websrv
{
    proxy_pass http://192.168.8.237:8080/;
}
root@GL-MT300N-V2:/etc/nginx/gi-conf.d# service nginx stop
root@GL-MT300N-V2:/etc/nginx/gi-conf.d# service nginx start
root@GL-MT300N-V2:/etc/nginx/gi-conf.d#
```

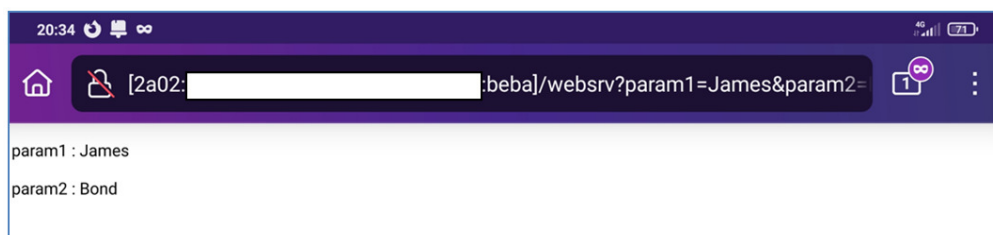
Et là Ô miracle :

⁶⁹ Le nom **websrv** est arbitraire mais ne doit pas interférer avec les dossiers utilisés par le Panneau d'Administration et LuCI.

⁷⁰ Par copier-coller (Maj-insert) par exemple.

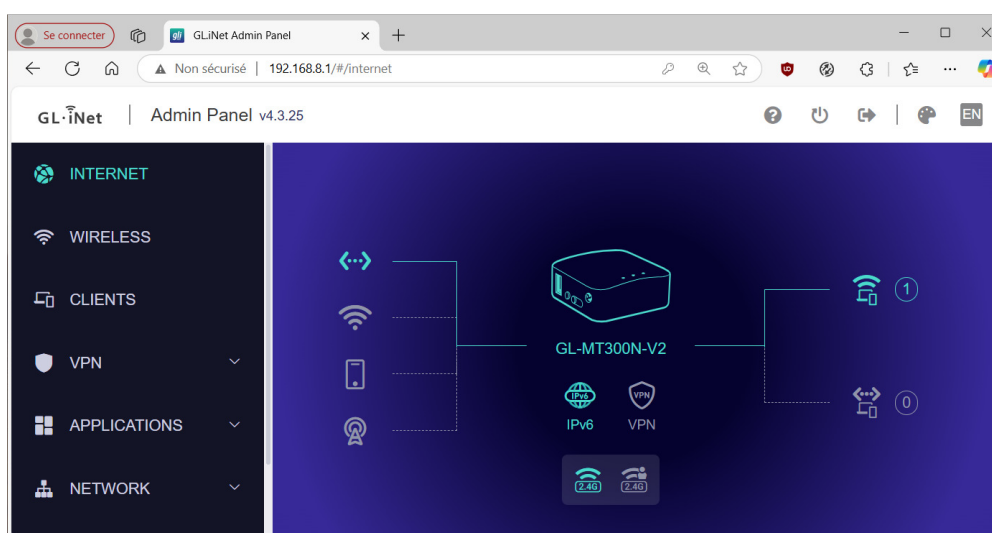
- On peut accéder à distance au site Web sur le PC avec l'URL suivante sur le Smartphone fonctionnant en données mobiles :

[http://\[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:beba\]/websrv?param1=James¶m2=Bond](http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:beba]/websrv?param1=James¶m2=Bond)



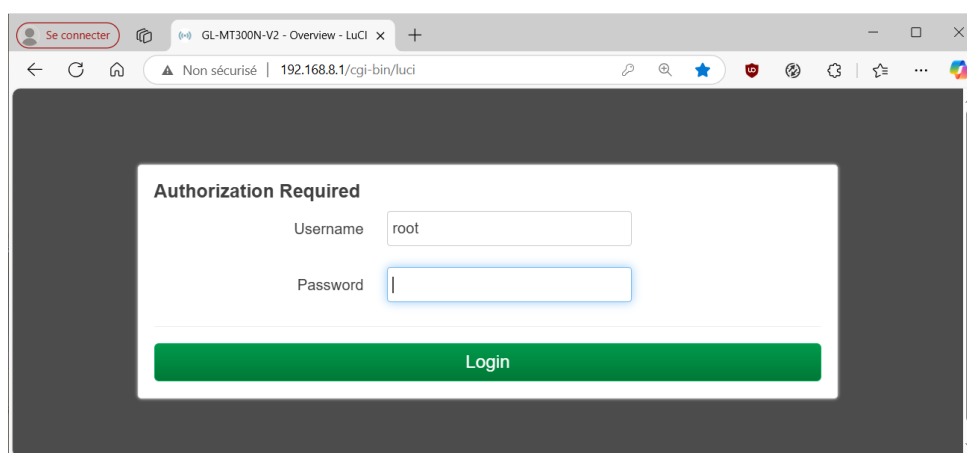
- Le Panneau d'administration du routeur est toujours accessible à partir d'un poste de son sous-réseau local avec l'URL :

<http://192.168.8.1>



- LuCI est toujours accessible à partir d'un poste de son sous-réseau local avec l'URL :

<http://192.168.8.1/cgi-bin/luci>



4 En résumé

Ce document a présenté toutes la démarche intellectuelle qui nous a conduits à résoudre le problème posé initialement, ceci grâce à 3 clés :

- Le protocole IPv6 qui permet de joindre à distance un poste sur un réseau privé.
- Un proxy inverse tel que NGINX pour effectuer localement la conversion HTTP-IPv6 => HTTP-IPv4.
- Le routeur multi protocole GL-MY300N-V2 sur lequel NGINX fonctionne par défaut en tant que serveur Web, où il a suffit d'étendre ses fonctionnalités.

A l'attention de ceux qui souhaiteraient adapter rapidement ce travail à leur contexte particulier, en voici le résumé :

Le problème de l'accès distant à un serveur HTTP-IPv4 situé sur le réseau local défini par une box Internet IPv6-IPv4 CGNAT a été résolu avec les étapes suivantes :

- Ajouter un routeur GL-MT300N-V2 dans ce réseau local, fonctionnant en mode routeur et créant ainsi un sous-réseau local du précédent (§ 3.1.1).
- Paramétrer le routeur pour qu'il fonctionne en IPv4 et IPv6 et rendre statiques les adresses IP attribuées par les services DHCP de la box (§ 3.1.3).
- Intégrer le serveur HTTP-IPv4 dans le sous-réseau local du routeur et rendre statique l'adresse IPv4 attribuée au poste serveur par le service DHCP IPv4 du routeur (§ 3.1.4).
- Dans le pare-feu du routeur, ajouter une autorisation d'accès en IPv4 et IPv6 sur son port 80 à partir de n'importe quelle zone (§ 3.1.4).
- Dans le pare-feu de la box, ajouter une règle d'accès au port 80 du routeur, en TCP-IPv6 (§ 3.1.4).
- Activer la fonction proxy inverse du service nginx sur le routeur, pour convertir une requête HTTP-IPv6 au routeur en requête HTTP-IPv4 au serveur local, en ajoutant dans `/etc/nginx/gl-conf.d` sur le routeur, un fichier de configuration (extension `.conf`) contenant des instructions de la forme :

```
location /webserv
{
    proxy_pass http://<adresse IPv4 du serveur>:<port du serveur>;
}
```

(§ 3.2).

Dès lors, l'accès au serveur local HTTP-IPv4 pourra se faire à distance, avec une URL de la forme :

[http://\[<adresse IPv6 du routeur>\]/webserv ?<paramètres>](http://[<adresse IPv6 du routeur>]/webserv ?<paramètres>)

Remarques

- Pour simplifier la présentation on n'a pas abordé ici le problème de la sécurité des accès distants HTTP-IPv6 au routeur. Il ne devrait pas être bien difficile à résoudre car, sur le routeur, le service **nginx** accepte des accès cryptés en SSL, comme on peut le constater dans le fichier de configuration **gl.conf**, qui contient les instructions suivantes (§ 3.2) :

```
listen 443 ssl;
listen [::]:443 ssl;
```

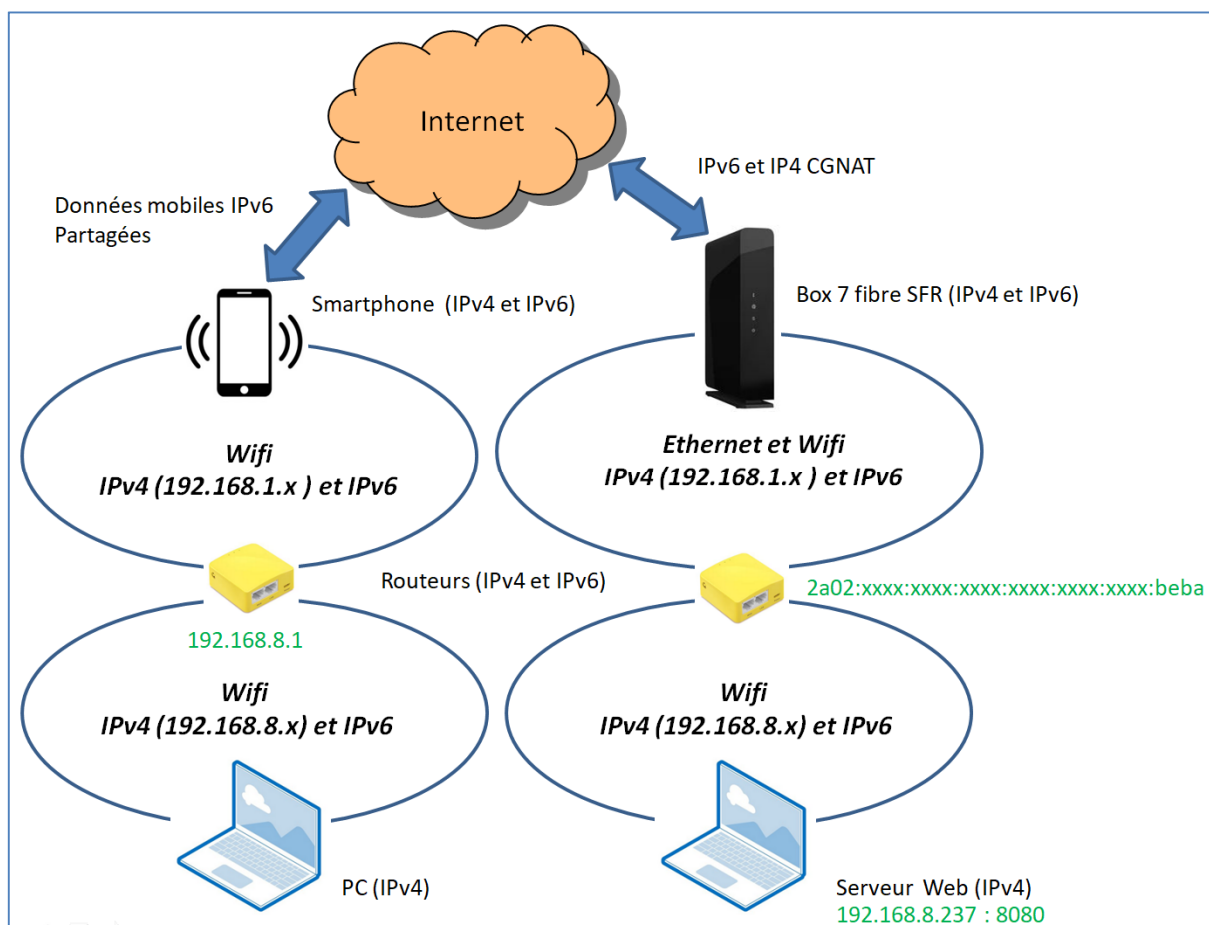
- Si l'utilisation du protocole IPv6 sur un poste distant pose problème, on peut là encore imaginer une solution analogue à celle ici mise en œuvre, mais en utilisant cette fois NGINX comme proxy direct dans le réseau local distant, de manière à effectuer la conversion HTTP-IPv4 => HTTP-IPv6. Là encore on peut le faire, en utilisant un deuxième routeur GL-MT300N-V2, fonctionnant également en mode IPv4 et IPv6, en ajoutant un fichier de configuration (extension .conf) dans /etc/nginx/gl-conf.d, mais avec les instructions suivantes où figure l'adresse IPv6 du premier routeur :

```
location /websrv
{
    proxy_pass http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:beba]/websrv/;
}
```

Ainsi, à partir d'un poste intégré dans le sous-réseau local défini par ce deuxième routeur, mais ne fonctionnant qu'en IPv4, on pourra accéder à distance au serveur Web du PC avec une URL de la forme :

<http://192.168.8.1/websrv?<paramètres>>

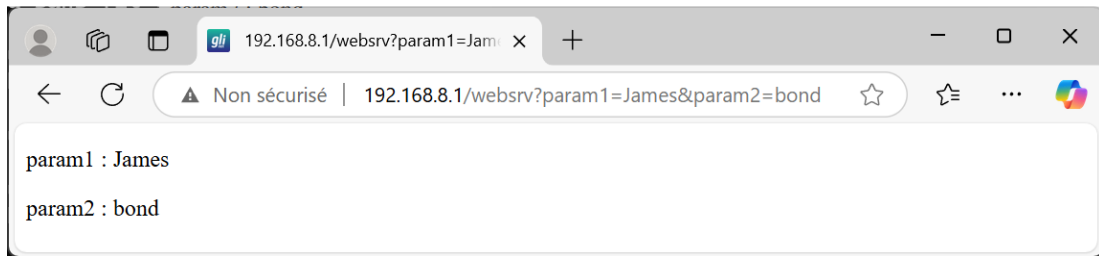
On peut le tester par exemple en connectant le Smartphone à Internet , toujours via ses données mobiles, mais en partageant cette fois sa connexion. Le deuxième routeur y sera connecté en Wifi, mais fonctionnera en mode routeur (cf. § 3.1.3). Il définira donc bien un sous-réseau local, 192.168.8.x en IPv4, dans lequel on intégrera un deuxième PC ne fonctionnant qu'en IPv4 (cf. § 3.1.2) :



Dès lors, l'URL suivante sur un navigateur Internet de ce deuxième PC :

<http://192.168.8.1/websrv?param1=James¶m2=Bond>

conduira à la page web attendue :



5 Application au système photovoltaïque

Pour résoudre le problème de l'accès distant de PVMonitor à PVRouter (cf. § 1), il suffit de remplacer le routeur **TP-Link TL802N** par le routeur **GL.iNet GL-MT300N-V2**.

Comme le précédent, il sera connecté à la box par un câble Ethernet mais fonctionnera en IPv4 et IPv6 (cf. § 3.1.3). De ce fait la box pourra lui attribuer des adresses IPv4 et IPv6 statiques, en paramétrant ses services DHCP (cf. § 3.1.3) :

- 192.168.1.34
- 2a02:xxxx:xxxx:xxxx:xxxx:xxxx:beba

Le routeur définira cette fois un sous-réseau Wifi IPv4 192.168.8.x, dans lequel on intégrera l'onduleur (192.168.8.100) et PVRouter (192.168.8.101), avec là encore des adresses statiques obtenues cette fois en paramétrant les services DHCP du routeur (cf. § 3.1.4).

On ajoutera les règles suivantes dans les pare-feux (§ 3.1.4) :

- Dans la box, autoriser l'accès au port 80 du routeur, en TCP-IPv6.
- Dans le routeur, autoriser l'accès en IPv4 et IPv6 sur son port 80 à partir de n'importe quelle zone.

Enfin, on activera la fonction proxy inverse du service nginx sur le routeur, en ajoutant dans `/etc/nginx/gl-conf.d`, un fichier de configuration (extension `.conf`) contenant les instructions suivantes (cf. § 3.2) :

```
location /websrv
{
    proxy_pass http://192.168.8.101:8080/;
}
```

5.1 Accès local de PVMonitor à PVRouter

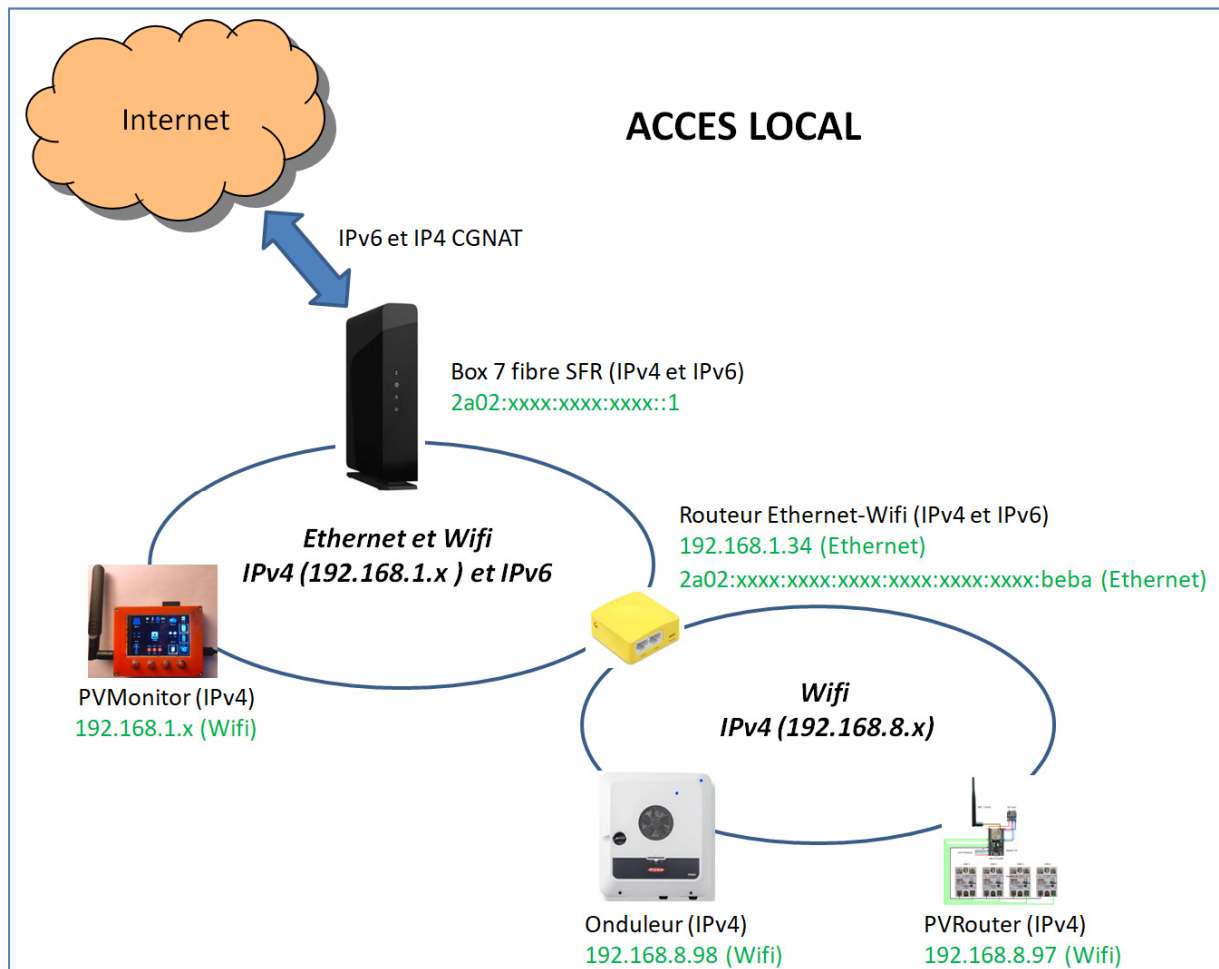
Comme c'est le cas actuellement, PVMonitor sera intégré dans le réseau local défini par la box, à laquelle il se connectera en Wifi IPv4 ; il recevra donc une adresse IPv4 de la forme 192.168.1.x.

Dès lors, pour accéder à PVRouter, il lui suffira de formuler des requêtes HTTP GET, avec des URL de la forme :

<http://192.168.1.34/websrv/data?Temperature=25&Battery=2&Boiler=2&HPump=2&SPPump=2>

Elles seront traduites par le proxy inverse en :

<http://192.168.8.101:8080/data?Temperature=25&Battery=2&Boiler=2&HPump=2&SPPump=2>



5.2 Accès distant de PVMonitor à PVRouter

Disposant d'un point d'accès Wifi à Internet, fonctionnant en IPv4 et IPv6⁷¹, on y connectera un second routeur **GL.iNet GL-MT300N-V2**, fonctionnant également IPv4 et IPv6 (cf. § 3.1.3), mais sur lequel on activera la fonction proxy direct du service nginx, en ajoutant dans `/etc/nginx/gl-conf.d`, un fichier de configuration (extension `.conf`) contenant les instructions suivantes (cf. § 4) :

```
location /websrv
{
    proxy_pass http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:beba]/websrv/;
}
```

Cette fois, PVMonitor sera connecté en Wifi IPv4 à ce second routeur et pour accéder à PVRouter, il lui suffira de formuler des requêtes HTTP GET, avec des URL de la forme :

<http://192.168.8.1/websrv/data?Temperature=25&Battery=2&Boiler=2&HPump=2&SPPump=2>

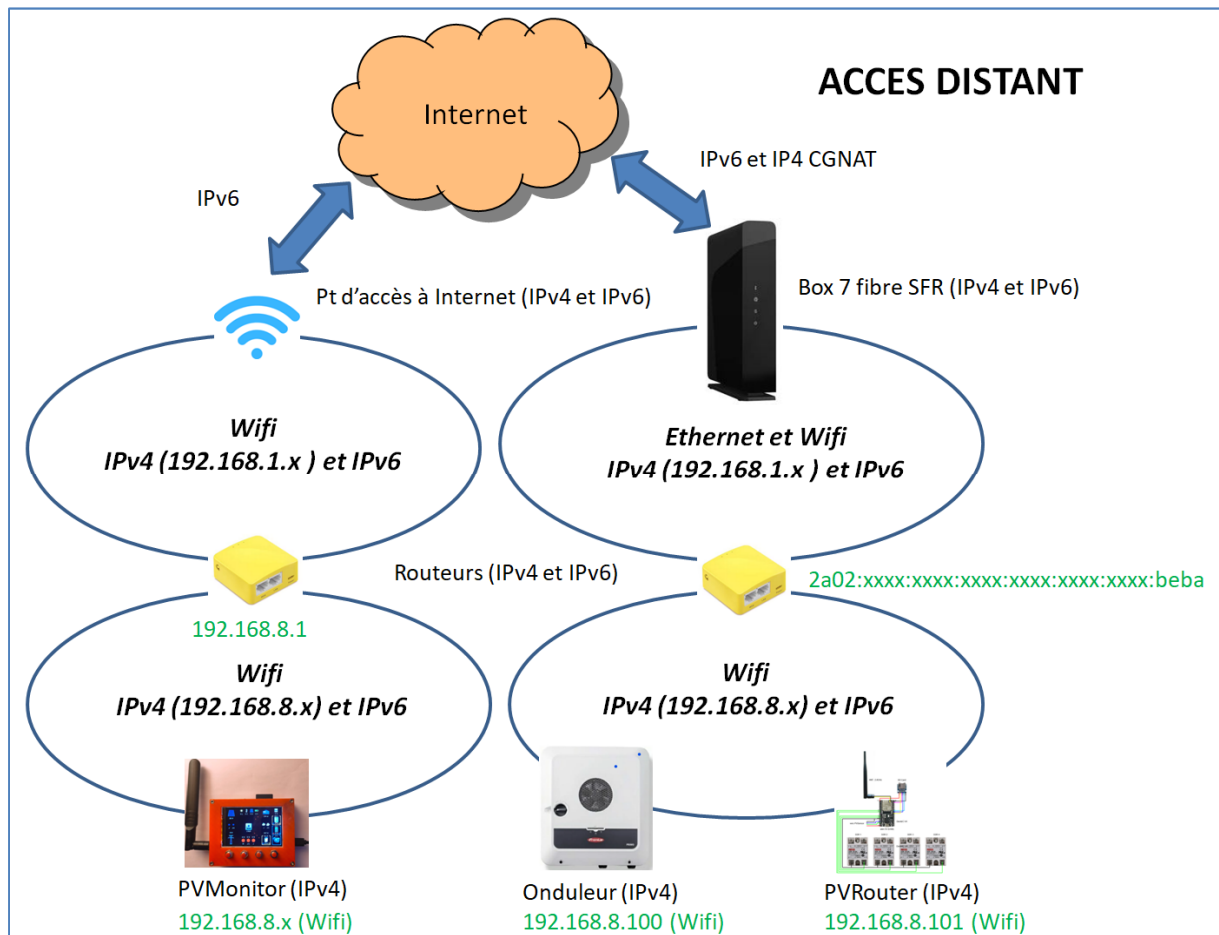
Elles seront traduites par le proxy direct du deuxième routeur en :

[http://\[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:beba\]/websrv/data?Temperature=25&Battery=2&Boiler=2&HPump=2&SPPump=2](http://[2a02:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:beba]/websrv/data?Temperature=25&Battery=2&Boiler=2&HPump=2&SPPump=2)

puis, par le proxy inverse du premier routeur en :

<http://192.168.8.101:8080/data?Temperature=25&Battery=2&Boiler=2&HPump=2&SPPump=2>

⁷¹ Box ou smartphone partageant une connexion à Internet via ses données mobiles par exemple.



Grâce à ces 2 routeurs, on peut donc continuer à opérer en IPv4 sur PVMonitor, pour faire un accès distant à PVRouter.

6 Annexes

6.1 Principales commandes SSH dans Putty

tab	Compléter la commande
↑	Rappeler la commande précédente
ctrl-c	Interrompre commande
cd <nom dossier>	Changer le dossier courant
ls	Liste des fichiers et des sous-dossiers du dossier courant
ls -alh	Liste verticale intelligible...
mkdir <nom sous-dossier>	Créer un sous-répertoire
rm <nom fichier>	Supprimer un fichier
rm -rf <nom dossier>	Supprimer un dossier
cat <nom fichier>	Afficher un fichier
mv <nom fichier> <nouv. nom fichier>	Renommer un fichier ou un sous-dossier
vi <nom fichier>	Editer un fichier dans vi
exit	Fermer Putty
clear	Effacer l'écran de Putty
service	Liste des services activés
service <nom service> start stop	Lancement Arrêt d'un service
reboot	Effectue un Reboot du routeur

6.2 L'éditeur Vi

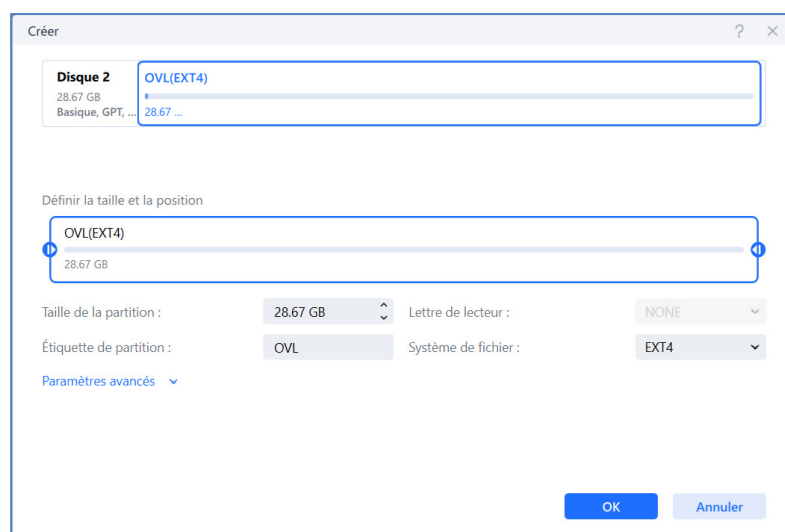
Touches ↑↓→←	Déplacer le curseur
i	Mode Insertion
Touche esc	Sortir du mode Insertion (*)
:w	Sauvegarder
:q	Quitter vi
:wq	Sauvegarder et quitter vi
vi <nom fichier>	Créer (s'il n'existe pas) un fichier dans le dossier courant et l'ouvrir dans vi
:q !	Quitter vi sans sauvegarder
dd	Supprimer la ligne du curseur
Touche delete	Supprimer le caractère du curseur (*)
Touche backspace	Supprimer le caractère avant le curseur (*)

(*) En mode Insertion

6.3 Augmentation de la mémoire flash du routeur GL-MT300N-V2, en utilisant une clé USB

Bien que non utilisée ici puisqu'il n'a pas été nécessaire d'ajouter une nouvelle application sur le routeur, nous croyons intéressant de présenter cette technique très utile, car elle est souvent décrite de manière trop complexe^{72 73}.

1. Formater une clé USB en **Ext4**, avec **EaseUS Partition Master**⁷⁴ par exemple. On utilisera ici une clé de **32GB** et on la nommera **OVL** dans ce qui suit.



2. Sur le routeur, dans **LuCI**, utiliser l'item de menu **System > Software** afin d'afficher la quantité de mémoire disponible pour les applications (1,29MB ici) :



3. Connecter la clé USB au routeur et cliquer sur l'item de menu **System > Mount Points** :

⁷² https://openwrt.org/docs/guide-user/additional-software/extroot_configuration

⁷³ <https://openwrt.org/docs/guide-user/storage/usb-drives-quickstart>

⁷⁴ <https://www.easeus.fr/partition-manager/partition-master-free.html>

Mounted file systems

Filesystem	Mount Point	Available	Used	Unmount
/dev/root	/rom	0 B / 11.75 MiB	100.00% (11.75 MiB)	-
tmpfs	/tmp	58.86 MiB / 59.55 MiB	1.16% (708.00 KiB)	-
/dev/mtdblock6	/overlay	1.29 MiB / 1.63 MiB	20.43% (340.00 KiB)	-
overlayfs:/overlay	/	1.29 MiB / 1.63 MiB	20.43% (340.00 KiB)	-
tmpfs	/dev	512.00 KiB / 512.00 KiB	0.00% (0 B)	-
/dev/sda1	/mnt/OVL	26.79 GiB / 28.24 GiB	0.00% (8.00 KiB)	Unmount

Mount Points

Mount Points define at which point a memory device will be attached to the filesystem

Enabled	Device	Mount point	Filesystem	Mount options	Run filesystem check
This section contains no values yet					

[Add](#)

4. Cliquer sur [Add](#) et choisir :

Mount Points - Mount Entry

[General Settings](#) [Advanced Settings](#)

Enabled ☒

UUID
 ⓘ If specified, mount the device by its UUID instead of a fixed device node

Mount point
 ⓘ Specifies the directory the device is attached to

[Dismiss](#) [Save](#)

5. Cliquer sur [Save](#) puis sur [Save & Apply](#) :

Mount Points

Mount Points define at which point a memory device will be attached to the filesystem

Enabled	Device	Mount point	Filesystem	Mount options	Run filesystem check	
☒	UUID: b891587e-4b17-4673-71d7-b45f48c4d777 (/dev/sda1, 28.67 GiB)	/overlay	auto (ext4)	defaults	No	Edit Delete

[Add](#)

SWAP

If your physical memory is insufficient unused data can be temporarily swapped to a swap-device resulting in a higher amount of usable RAM. Be aware that swapping data is a very slow process as the swap-device cannot be accessed with the high data rates of the RAM.

Enabled	Device
This section contains no values yet	

[Add](#)

[Save & Apply](#) [Save](#) [Reset](#)

- En utilisant l'item de menu **System > Reboot**, rebooter le routeur.
- Après son redémarrage, relancer **LuCI** et utiliser l'item de menu **System > Software** afin d'afficher la quantité de mémoire disponible pour les applications (28,24GB maintenant) :

Software

Free space:
99% (28.24 GiB)